ASMPT

TU/e EINDHOVEN UNIVERSITY OF TECHNOLOGY

Department of Electrical Engineering

Model Augmentation for a High-Precision Dual-Drive Gantry

by

D.J.L.M. van den Berg

MSc Thesis

Thesis committee

dr. ir. M. Schoukens
prof. dr. ir. R. Tóth
dr. ir. M.F. Shakib
ir. J. W. Gerritsen

Graduation

Program:	Artificial Intelligence and Engineering Systems
Research group:	Control Systems
Thesis supervisor:	dr. ir. M. Schoukens
Date of defense:	17-11-2026
Student ID:	1555561
Study load (ECTS):	45
Track:	High-Tech Systems and Robotics

The research of this thesis has been carried out in collaboration with ASMPT, Beuningen.

This thesis is public and Open Access.

This thesis has been realized in accordance with the regulations as stated in the TU/e Code of Scientific Conduct.

Disclaimer: the Department of Electrical Engineering of the Eindhoven University of Technology accepts no responsibility for the contents of MSc theses or practical training reports.

Model Augmentation for a High-Precision Dual-Drive Gantry

Dirk van den Berg
Control Systems Group, Dept. of Electrical Engineering
Eindhoven University of Technology
Eindhoven, The Netherlands
In collaboration with *ASMPT*, Beuningen, The Netherlands

Supervisors: dr. ir. M. Schoukens, prof. dr. ir. R. Tóth, ir. J.H. Hoekstra (TU/e);
ir. Q. van den Elsen, ir. J. W. Gerritsen (ASMPT)

***Abstract*—A plant model can be used to evaluate a changed feedback controller before it is tested on the machine. For this, both predictive accuracy and physical interpretability are required. Physics-based models do not capture all dynamics, while refitting their parameters or adding an unconstrained learned component can absorb dynamics the baseline already describes. We augment a physics-based model of a dual-drive gantry in parallel with a neural network and added states, with a predictive-state initialisation, trained in closed loop with the known controller. To keep the physical parameters interpretable, we extend orthogonal-by-construction (OBC) augmentation to a state-space baseline that is nonlinear in its parameters. The method is evaluated in simulation on held-out positions and unseen motion profiles, against the baseline and a black box model. The augmented model improves the prediction where the data excite the missing dynamics, but the parameter estimates still depend on where training starts.**[own]

***Index Terms*—**

I. Introduction

[Changes: black = original, blue = original, moved here (from … end), green = changed or new, ~~struck~~ = deleted ("moved to" = moved, not deleted), superscript = comment id.]

Semiconductor devices are ~~used in technologies~~ essential for applications[M-p2b] such as artificial intelligence, 5G and autonomous vehicles. Shortages can affect entire industries and national security [1]. Manufacturing these devices requires high-precision motion systems, with strict requirements on alignment accuracy, yield, throughput and cost [2], [3]. [M-p2b] Accurate plant models help to meet these requirements [4], [5].[M-p2b] This thesis focuses on ~~a dual-drive gantry motion system~~ one such motion system: a dual-drive gantry[own] in which two parallel linear actuators support a common beam carrying the payload. ~~The gantry positions this payload at successive placement locations.~~[own] The shared beam couples the dynamics of both actuators, so the motion of one actuator influences the otherand therefore affects its tracking accuracy[M-p2b]~~, and the system~~ [6], [7]. The system[own] exhibits position-dependent behaviour ~~[6], [7]~~[own]and flexible dynamics [6]. ~~This position dependence can be described by a linear parameter-varying (LPV) model.~~[Dirk] ~~Furthermore, cross-coupling means that the motion of one actuator can disturb the other, which affects the tracking behaviour during simultaneous motion.~~[M-p3b]

A plant model can be used to evaluate a changed feedback controller before it is tested on the machine. A closed-loop simulation can be faster than an experiment, allowing multiple ~~controller~~ controllers[own] to be compared simultaneously, and without risk of harming the hardware. ~~Such models are also valuable for root cause analysis.~~[moved to end of P2, Dirk] ~~This requires a model with physically meaningful parameters, because a change in machine behaviour can then be traced back to a physical cause, such as a change in stiffness or damping.~~[M-p2c from P4, own] For the plant model of the dual-drive gantry, both predictive accuracy and physical interpretability are required. ~~Physical interpretability means that~~ [own]In this thesis, physical interpretability means that the identifiable combinations of the physical parameters ~~(Section III-C)~~[M-mail] are estimated uniquely [8]. This does not guarantee identifying the true parameters, but a deviation ~~from the trained model~~[M-p2c] can then indicate which physical parameter might have changed~~.~~ ,[M-p2c end from above, Dirk] ~~Such models are therefore also~~ making such models[M-p2c] valuable for root cause analysis [9]. [**TODO:** Confirm the supporting passages in Isermann 1993, PDF pp. 3 and 12.][end] ~~Both uses require an accurate representation of the underlying plant dynamics.~~[M-p2c]

White-box models are attractive because they reflect the physics of the system, which makes them interpretable. First-principles (FP) models are a specific type of white-box model, derived from physical laws and described by quantities such as mass, stiffness, and damping. However, purely physics-based models do not always capture all behaviour needed for accurate prediction across the operating range [10]. More accurate FP models can be developed, but this is labour-intensive [5]. [M-p2d]Purely data-driven (black-box) models such as neural-network state-space models can describe measured behaviour well, but can offer limited physical insight and are not guaranteed to be accurate at operating conditions not covered by the training data [5], [10]. ~~These models can also require large amounts of data to train [10], [11].~~[M-p2d] In addition, these models need to learn behaviour that is already known from first principles, such as the rigid-body dynamics [5].

Physics-informed approaches, ~~including physics-informed neural networks (PINNs)~~ such as physics-informed or physics-guided neural networks and Gaussian-process-based methods[M-p2e], incorporate physical ~~laws as constraints~~ knowledge[M-p2e] during training [11], [12], [10][M-p2e]. ~~However, these methods require the physics equations to be known and~~ , but[M-p2e] the physics-informed and physics-guided neural networks discussed by Hoekstra et al. still rely on a black-box model for the entire system [5]. ~~For the plant model of the dual-drive gantry, both predictive accuracy and physical interpretability are required. Physical interpretability means that the identifiable combinations of the physical parameters are estimated uniquely [8]. This does not guarantee identifying the true parameters, but a deviation from the trained model can then indicate which physical parameter has changed.~~[moved to P2, own] ~~However, purely physics-based and purely data-driven models can each have limitations in meeting both requirements [10].~~[M-p2e]

Grey-box modelling is the logical choice, because it is able to combine a baseline model with interpretable parameters ~~with data-driven flexibility needed~~ and a data-driven model[Dirk] for accurate prediction [13]. Grey-box approaches include parameter re-estimation within the physics model structure, static residual correction on the model output, and dynamic augmentation that extends the baseline with learned states [5], [10]. Model augmentation combines a physics-based baseline model with a learned component, so that the learned component only needs to capture the dynamics the physics-based model cannot describe [5], [14][own]. Dynamic augmentation is chosen ~~to be able to model missing dynamics, such as flexible dynamics. These~~ to model missing dynamics that[own] require additional states, which parameter re-estimation and static output correction cannot provide [5]. However, related work ~~at ASMPT~~[Jan1] on ~~a~~ an industrial[Jan1] wire bonder shows that preserving physical interpretability during model updating remains an open challenge [4]. Refitting the physical parameters allows them to ~~absorbs~~ absorb[own] unmodelled dynamics, and an unconstrained ~~learning~~ learned[own] component is able to absorb dynamics the baseline already describes [8]. ~~Learning-based hybrid modelling has already been demonstrated using measurements from an industrial robot [14].~~[own] ~~However, a key gap remains: there is no grey-box modelling framework for the dual-drive gantry that simultaneously addresses coupled multi-input multi-output (MIMO) dynamics, position-dependent behaviour, closed-loop data, and preserves physical interpretability.~~[moved to end of P6, own] ~~Model augmentation using a linear fractional representation (LFR)~~ Dynamic augmentation[Jan26] has already been demonstrated, but Hoekstra et al.'s examples do not consider position-dependent baselines or ~~closed-loop data~~ training with a simulated feedback controller[own] [5]. The related work ~~at ASMPT~~ on the wire bonder[Jan1] uses closed-loop data, but keeps the physical parameters fixed during training [4]. Orthogonal-by-construction (OBC) augmentation keeps the learned component orthogonal to the baseline regressor space, so that the physical parameter estimates are unique under the stated conditions [8]. Györök et al. formulate this for discrete-time input-output models, leaving extension to state-space models for future work. [from above, own] ~~However, a~~ A[own] key gap remains: the reviewed approaches do not ~~for the dual-drive gantry~~[own] simultaneously address coupled multi-input multi-output (MIMO) dynamics, position-dependent behaviour~~, closed-loop data, and preserves~~ and closed-loop data while preserving[own] physical interpretability.[end] This thesis therefore develops such a framework [from gap, own] for the dual-drive gantry[end] and evaluates it in simulation. ~~The augmented model should predict more accurately than the physics-based baseline across operating points and motion profiles, while its physical parameters remain interpretable.~~[M-p3b]

This thesis addresses the following research question: *For the dual-drive gantry, how should a physics-based plant model be augmented to capture unmodelled dynamics, improving prediction accuracy across operating points and motion profiles while preserving the physical interpretability of its parameters?* To answer this research question, the following three aspects are investigated:

1) Which augmentation structure and trainingmethod[own] let the learned component capture the missing dynamics from closed-loop data?
2) To what extent does orthogonal-by-construction augmentation keep the physical parameters interpretable, compared to unconstrained estimation?
3) How does the augmented model generalise to held-out operating points and unseen motion profiles, compared to ~~a~~ the physics baseline and a[Q13] black-box model?

~~The paper is organised as follows: first, the identification problem is stated in Section II, followed by the physics baseline in Section III. Then, Section IV introduces the augmented model and its identification in closed loop, and Section V the orthogonal construction. The experiment design is described in Section VI, and the results are given and discussed in Section VII. The conclusions are given in Section VIII.~~[M-p3] The thesis is organised as follows: Section II states the identification problem, Section III the physics baseline, Section IV the augmented model and its closed-loop identification, Section V the orthogonal construction, Section VI the experiment design, Section VII the results and discussion, and Section VIII the conclusions.[M-p3] [**Note:** Aspect 1 (LPV versus LTI) is removed: the LPV form follows from the equations of motion (Section III-A), so it is a modelling choice rather than a research result. The LTI comparison (on X the LTI error is 1.5 to 3.7 times the LPV error) moves to the appendix. Aspects 2 to 4 are reworded to the final method.]

II. Identification Problem

[Changes: black = original, blue = original, moved here (from … end), green = changed or new, ~~struck~~ = deleted ("moved to" = moved, not deleted), superscript = comment id.]

~~The dual-drive gantry that we study in this thesis is a high-precision motion system of ASMPT, used in back-end semiconductor assembly and packaging equipment.~~[ASMPT] ~~The gantry~~ We consider a dual-drive gantry with[ASMPT] ~~two actuators~~

two parallel linear actuators[Js21] that are connected with a cross-arm and a third linear [Js21]actuator that moves a payload along this arm (Fig. 1), as in García-Herreros et al. [6][own]. [**Note:** Kan specifiek asmpt attributen voor gebruiken garcia in de matlab simulatie in principe niks gebruikt wat niet in garcia staat, met euler lagrange] The actuator forces are defined as $u_{\text{act}} = [F_{X1}\ F_{X2}\ F_Y]^\top$ and the measured positions $q_{\text{act}} = [X_1\ X_2\ Y]^\top$. Here F_{X1}, F_{X2} and F_Y are the forces and X_1, X_2 and Y the measured positions of the ~~two X actuators and the Y actuator~~ two X and the Y actuators[Dirk], each from a linear encoder [6][Q17].

The system to be identified is the gantry system described ~~as,~~ by

$$\dot{x}(t) = f\big(x(t), u_{\text{act}}(t)\big), \qquad y_k = q_{\text{act}}(kT_s) + v_k, \quad (1)$$

where the ~~state x is unknown, except that it contains the actuator positions q_{act}~~ state x contains the actuator positions q_{act}[own], y_k are the measured positions at sample k with sampling time T_s, and v_k is zero-mean, coloured measurement noise. The ~~state-transition function f is~~ function f and the dimension of x are[own] unknown. The system runs in closed loop with an assumed known linear time-invariant feedback controller ~~(Section IV-B)~~[M-p3], which gives the data set $\mathcal{D}_N = \{u_{\text{act},k}, y_k\}_{k=1}^N$. The controller computes the actuator forces from the error between the reference r_k and the measured positions y_k, and a feedforward u_k^{ff} is added (Fig. 2):[M-p3]

$$\begin{aligned} x_{k+1}^{\text{fb}} &= A_{\text{fb}} x_k^{\text{fb}} + B_{\text{fb}}(r_k - y_k), \\ u_{\text{act},k} &= C_{\text{fb}} x_k^{\text{fb}} + D_{\text{fb}}(r_k - y_k) + u_k^{\text{ff}}, \end{aligned} \quad (2)$$

[from IV-B, M-p3] where $(A_{\text{fb}}, B_{\text{fb}}, C_{\text{fb}}, D_{\text{fb}})$ are the state-space matrices of the controllerK_{fb}[own], with state x^{fb},[M-p3] discretised with sampling time T_s, and the actuator forces are held constant between samples, $u_{\text{act}}(t) = u_{\text{act},k}$ for $kT_s \le t < (k+1)T_s$[own]. Both r_k and u_k^{ff} are in actuator coordinates.[end] The noise v is

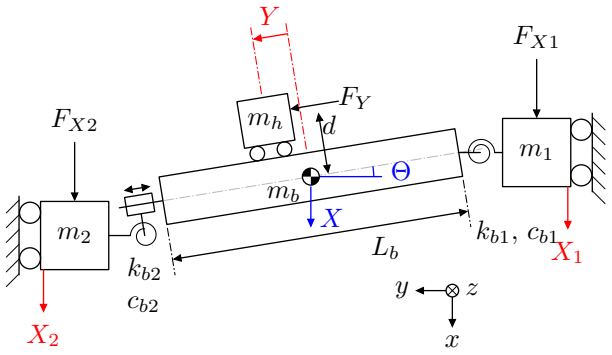


Fig. 1: Lumped-parameter model of the Dual-drive gantry stage, adapted from [6]. At X_2, a prismatic joint lets the cross-arm slide, allowing it to rotate. Red labels are actuator coordinates and blue labels generalised coordinates, with Y in both.

[**TODO:** Will need to add springs to the current torsion; feedback from Jasper. Move X1 a bit down. Jasper said I should keep both torsion but i can add the primsatic one but need to explain that it is that.]

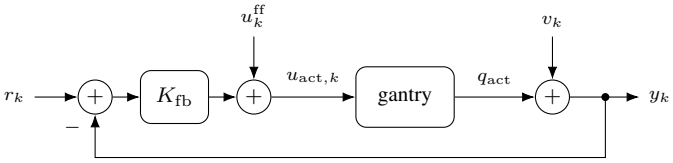


Fig. 2: Closed loop that generates the data: the controller K_{fb} of (2), the feedforward u_k^{ff}, the gantry (1) and the measurement noise v_k.

independent of r and u^{ff}. The force u_{act} is correlated with v, because the controller computes it from y.[M-p3]

We use the lumped-parameter model of Fig. 1 as our physics-based baseline model of (1). This model does not capture all dynamics of the system. We identify these missing dynamics from $\mathcal{D}_N$ and augment the baseline with learned components and additional states [5].

The goal is to estimate the physical and learned parameters of the augmented model from $\mathcal{D}_N$, such that it captures the missing dynamics and accurately predicts the system, while the identifiable combinations of the physical parameters are estimated uniquely. We use the SUBNET method of Beintema et al. [15] to train the model on short windows of $\mathcal{D}_N$, where each window is simulated with the controller K_{fb}. The initial states are estimated by an encoder network from past inputs and outputs.

III. Dual-Drive Gantry Physics Baseline

In this section, we ~~give~~ state[Dirk] the equations of motion of the baseline ~~(Section III-A)~~[own], its ~~position-dependent representation~~ position-dependent state-space model[own] ~~(Section III-B)~~[own], and the identifiable combinations of ~~its~~ the[Dirk] physical parameters ~~with their parametrisation~~ and how we parametrise them[Dirk] for estimation ~~(Section III-C)~~[own].

A. Equations of motion

~~The physics baseline of the gantry system used in this thesis is the lumped-parameter model of the dual-drive gantry of García-Herreros et al. [6].~~[Dirk] García-Herreros et al. derive the equations of motion of the model in Fig. 1 from the Euler-Lagrange equations [6, Eqs. (3) to (8)]. Let L_b denote the cross-arm length between the two flexible joints. The generalised coordinates $q = [X\ \Theta\ Y]^\top$ are defined as $X = (X_1 + X_2)/2$ and $\Theta = (X_1 - X_2)/L_b$, the position and yaw angle of the cross-arm, while Y is unchanged. These coordinates avoid closed kinematic chains when defining the kinematics [6]. They also show the coupling caused by the non-uniform load distribution between actuators X_1 and X_2, from the payload and differences in motor characteristics. The coordinate and force transformations are

$$q = P^{-\top} q_{\text{act}}, \qquad u = P u_{\text{act}}, \qquad P = \begin{bmatrix} 1 & 1 & 0 \\ L_b/2 & -L_b/2 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad (3)$$

where u denotes the generalised force. Following García-Herreros et al. [6], the baseline assumes $\cos(\Theta) \approx 1$ and $\sin(\Theta) \approx 0$, valid for small yaw angles, and assumes that the Coriolis and centripetal force terms are negligible compared to the inertia, damping, and stiffness forces. Our baseline model does not include the Coulomb friction of [6, Eq. (6)]. Instead, it is part of the missing dynamics of Section II. Under these assumptions, the baseline equations of motion are

$$M\big(Y(t)\big)\ddot{q}(t) + C\dot{q}(t) + Kq(t) = u(t), \tag{4}$$

with

$$M(Y) = \begin{bmatrix} m_1+m_2+m_b+m_h & \frac{L_b}{2}(m_1-m_2)-m_hY & 0 \\ \frac{L_b}{2}(m_1-m_2)-m_hY & J_b+J_h+\frac{L_b^2}{4}(m_1+m_2) & -m_hd \\ & +m_h(d^2+Y^2) & \\ 0 & -m_hd & m_h \end{bmatrix}, \tag{5a}$$

$$C = \begin{bmatrix} c_{g1}+c_{g2} & \frac{L_b}{2}(c_{g1}-c_{g2}) & 0 \\ \frac{L_b}{2}(c_{g1}-c_{g2}) & c_{b1}+c_{b2}+\frac{L_b^2}{4}(c_{g1}+c_{g2}) & 0 \\ 0 & 0 & c_y \end{bmatrix}, \tag{5b}$$

$$K = \begin{bmatrix} 0 & 0 & 0 \\ 0 & k_{b1}+k_{b2} & 0 \\ 0 & 0 & 0 \end{bmatrix}. \tag{5c}$$

From here on the notation for the time dependency is dropped. The masses, ~~flexible-joint~~ [Dirk] stiffness and damping coefficients, and dimensions L_b and d are shown in Fig. 1. The two X actuators have masses m_1 and m_2, the cross-arm m_b and the payload m_h. The yaw inertias of the cross-arm and payload are J_b and J_h. The actuator damping coefficients are c_{g1}, c_{g2} and c_y, while the flexible joints have stiffness k_{b1}, k_{b2} and damping c_{b1}, c_{b2}. The distance between the centres of mass of the payload and cross-arm is d [6]. The nominal values are taken from [6, Table I] and listed in Appendix A.1. ~~To prevent the definition of q from being parameter-dependent, the cross-arm length L_b is fixed at the nominal value listed in Appendix A.1 rather than treated as a free model parameter.~~ [moved to III-C, Js33] ~~Moving the payload position Y changes the mass distribution of the gantry and therefore its inertia matrix $M(Y)$. This position dependence is included by representing the baseline in linear parameter-varying (LPV) form, with the payload position Y as scheduling variable [16].~~ [moved to III-B] ~~Freezing $M(Y)$ at $Y = 0$ increases the simulation error on X by up to a factor 3.7 (Fig. ?? in Appendix ??).~~

B. Baseline state-space model

- Baseline in the state-space form of (1) (Section II)
- $M(Y) = M_0 + YM_1 + Y^2M_2$ (used by III-C)
- $M(Y) \succ 0$ for every Y, so $\ddot{q}$ is uniquely defined; explicit $\dot{\tilde{x}} = F(\tilde{x}, u)$
- With Y as scheduling variable this is a self-scheduled LPV model (Tóth, Drenth)
- Rational in Y through $M(Y)^{-1}$ and (6); exact LPV-LFR in Appendix A.2

[from III-A] Moving the payload position Y changes the mass distribution of the gantry and therefore its inertia matrix $M(Y)$. [end] ~~To separate the scheduling variable Y, deconstruct the inertia matrix (5a) into~~ The inertia matrix (5a) is [Dirk] a polynomial in Y,

$$M(Y) = M_0 + YM_1 + Y^2M_2, \tag{6}$$

~~M_0, M_1 and M_2 are~~ where M_0, M_1 and M_2 are [Dirk] given in Appendix A.2.

With positive masses and inertias, $M(Y) \succ 0$ for every $Y \in \mathbb{R}$ (Appendix A.3), ~~the baseline LFR is well posed throughout the operating range~~ we invert $M(Y)$ to calculate $\ddot{q}$ from (4) [Dirk].

~~For this realisation, well-posedness is equivalent to nonsingularity of $M(Y)$ (Appendix A.2).~~ [moved to A.2, Dirk] The augmentation of Section II assumes a baseline model in state-space form [5]. [M-mail from IV-A, Dirk] With the state $\tilde{x} = [q^\top \ \dot{q}^\top]^\top$, [Dirk] ~~Writing~~ writing [Dirk] (4) in state-space form gives the state derivative [end]

$$\dot{\tilde{x}} = F(\tilde{x}, u) = \begin{bmatrix} \dot{q} \\ M(Y)^{-1}\big(u - C\dot{q} - Kq\big) \end{bmatrix}. \tag{7}$$

~~where $u = Pu_{\text{act}}$ is the generalised force (3).~~ [own]

~~Because of $M(Y)^{-1}$, (7) is rationally dependent on Y.~~ [moved to below, own] ~~Treating Y as an externally specified scheduling signal, (7) is linear in $\tilde{x}$ and u, with matrices that depend on Y: it is a linear parameter-varying (LPV) model with Y as scheduling variable [16].~~ [own] This position dependence is included by representing ~~the baseline~~ (7) [own] in linear parameter-varying (LPV) form, with the payload position Y as scheduling variable [16]. For a given Y, (7) is then linear in $\tilde{x}$ and u. [own] The payload position Y is both a state and the scheduling variable, making the baseline a self-scheduled LPV model [17] [own]. Using the state Y as the scheduling variable, the LPV form gives the same equations of motion as (4) for any time-varying Y, without making an assumption on $\dot{Y}$. ~~Equation (7) defines the continuous-time baseline from which the discrete-time transition used in Section IV is obtained.~~ [M-mail]

Because of $M(Y)^{-1}$ and (6) [own], the matrices in (7) depend rationally on Y. We can also write the baseline as an exact linear fractional representation (LFR), with a linear time-invariant part interconnected with a scheduling block that depends on Y. Appendix A.2 gives the derivation and shows that the algebraic loop is well posed if $M(Y)$ is nonsingular. [Dirk] ~~A linear fractional representation (LFR) is introduced to separate this rational scheduling dependence exactly, with an LTI part whose matrices do not depend on Y. We move the dependence out of the LTI part G and into a Delta-block $\Delta(Y)$, with the scheduling being connected in feedback with G (Fig. 10) [5], [17]. Substituting (6) into (4) gives (7). Here Y only enters through $Y\ddot{q}$ and $Y^2\ddot{q}$, with $Y^2\ddot{q} = Y(Y\ddot{q})$. This lets us apply the same scheduling variable twice instead of introducing a second scheduling variable Y^2 (8), with $a_1 = Y\ddot{q}$ and $a_2 = Y^2\ddot{q}$. We treat a_1 and a_2 as independent signals, so (7) becomes (9), which contains no Y. This is allowed, because the Delta-block below imposes $a_1 = Y\ddot{q}$ and $a_2 = Ya_1$ again. The latent variables are $z = [\ddot{q}^\top \ a_1^\top]^\top$ and $w = [a_1^\top \ a_2^\top]^\top$, with the Delta-block mapping z to w (10). We write the LTI part G of the LFR in continuous time, adapting the discrete-time block structure of [?]Eq. (2)]drenth2026lpvlfr (11), where $\tilde{x} = [q^\top \ \dot{q}^\top]^\top$ is~~

~~the state and q the output. The first block row is the state equation, with $\ddot{q}$ obtained from (9) by multiplying with M_0^{-1}. For the blocks of G, see Appendix A.2. Conversely, closing the algebraic loop with (10) results in $a_1 = Y\ddot{q}$ and $a_2 = Y^2\ddot{q}$, which turns (9) back into (4). The LFR is therefore an exact realisation of the baseline. The LFR is well posed if the algebraic loop has a unique solution for every Y [?]Def. 1]drenth2026lpvlfr.~~[moved to A.2, Dirk]

C. Physical parameters and identifiable combinations

- With q measured, known force scale and full actuation, the input-output behaviour fixes M_0, M_1, M_2, C, K and conversely (this work)
- Twelve nonzero entries, three equal to m_h, leave ten independent functions: exactly ten combinations are structurally identifiable (this work; definition Ovchinnikov)
- Written physically as θ_{base}, the gantry counterpart of base parameters (Gautier and Khalil); one-to-one with the entries for $m_h > 0$ (this work)
- Four lost directions; estimates reported in θ_{base}; result independent of the encoder; augmented model in Section IV; recovery from the records in Setup
- Positive θ_{base} does not give $M(Y) \succ 0$; that holds iff (9) (this work; proof Appendix A.3)
- Training keeps (9) at every iterate; the map and its justification are in Section IV-C

The physical parameters are estimated jointly with the augmentation ~~(Section IV)~~[M-mail], and ~~interpretability requires that their estimates are unique (Section I)~~ we require that their identifiable combinations are estimated uniquely (Section II)[own]. ~~The fourteen parameters θ of Table VI cannot all be identified from input-output data.~~[moved to III-C, below, own] ~~The positions X_1, X_2, Y are measured, which gives q through (3), and the forces u are known in absolute scale.~~[own] [from III-A, Js33] To prevent the definition of q from being parameter-dependent, the cross-arm length L_b is fixed at the nominal value listed in Appendix A.1 rather than treated as a free model parameter.[end] [from III-C, above, own] The fourteenremaining[own] parameters of Table VIin Appendix A.1[M-p4], defined as θ,[own] cannot all be identified from ~~input-output data~~ the forces and positions in $\mathcal{D}_N$ (Section II)[M-p5].[end] ~~With unconstrained initial conditions and full actuation (P invertible), any acceleration $\ddot{q}$ can be reached at any state within the operating range.~~[moved to below, own] ~~Since q is measured, the state $(q, \dot{q})$ is observable.~~[own] Two parameter sets θ and θ' that produce the same ~~motion~~ positions q[M-p5] for every input force ~~satisfy~~[M-p5]also have the same $\dot{q}$ and $\ddot{q}$, since both are derivatives of q.[M-p5] Writing (4) for both sets and subtracting gives[M-p5]

$$\big(M(Y) - M'(Y)\big)\ddot{q} + (C - C')\dot{q} + (K - K')q = 0$$

where $M(Y)$, C and K are evaluated at θ and $M'(Y)$, C' and K' at θ'. [from above, own] With unconstrained initial conditions and full actuation (P invertible), any acceleration $\ddot{q}$ can be reachedby (7)[own] at any state ~~within the operating range~~[own].[end] Therefore the identity must hold for all these states and accelerations, ~~this~~ which[own] implies that $K = K'$, $C = C'$ and $M(Y) = M'(Y)$. Since $M(Y)$ is quadratic in Y, this also implies that $M_0 = M_0'$, $M_1 = M_1'$ and $M_2 = M_2'$. If these matrices are equal, the input-output behaviour is also the same. ~~Therefore, the part of θ that can be identified is determined by the parameter combinations in M_0, M_1, M_2, C and K.~~[own] ~~If we ignore the symmetric entries, (5) contains twelve nonzero entries. Three of these, $M_{0,33}$, $-M_{1,12}$ and $M_{2,22}$, are equal to m_h, where $M_{i,jk}$ denotes entry (j,k) of M_i. This leaves us with ten distinct functions of θ. The Jacobian of these ten functions with respect to $(m_h, d, m_1, m_b, J_b, c_{g1}, c_{g2}, c_{b1}, c_y, k_{b1})$ is block triangular and has a determinant of $m_h L_b^2/2 \neq 0$, meaning that the functions are independent.~~[moved to Appendix A.4, M-p5] Since the matrices ~~only~~[own]depend on ~~these ten functions~~ ten independent combinations of θ (Appendix A.4)[M-p5], ~~exactly ten independent combinations of θ~~ only these combinations[own] can be identified. Following the idea of base parameters used in robot dynamics [18],[own] ~~We~~ we[own] define ~~these ten combinations~~ them[own] as

$$\theta_{\text{base}} = [k_{b,\Sigma}, c_{g1}, c_{g2}, c_y, c_{b,\Sigma}, m_h, m_\Sigma, m_\Delta, J_{\text{eff}}, d]^\top, \quad (8)$$

with $k_{b,\Sigma} = k_{b1} + k_{b2}$, $c_{b,\Sigma} = c_{b1} + c_{b2}$, $m_\Sigma = m_1 + m_2 + m_b$, $m_\Delta = m_1 - m_2$ and $J_{\text{eff}} = J_b + J_h + \frac{L_b^2}{4}(m_1 + m_2)$. ~~This follows the idea of base parameters used in robot dynamics [18].~~[own] [TODO: check source] ~~For $m_h > 0$, θ_{base}~~ Since θ_{base}[own] can be uniquely determined from ~~the ten entries~~ the matrices[M-p5] (Appendix A.4). ,[own] ~~The four lost directions are $k_{b1} - k_{b2}$, $c_{b1} - c_{b2}$, $J_b - J_h$, and a mass shift from the carriages to the cross-arm at fixed J_{eff}.~~[own] ~~We therefore estimate~~ we estimate[own] θ_{base} instead of θ, ~~presented in the results~~.

~~Positive θ_{base} does not imply $M(Y) \succ 0$. For~~ In Section III-B, $M(Y) \succ 0$ follows from positive masses and inertias. For[own] $m_h > 0$, $m_\Sigma > 0$ and $J_{\text{eff}} > 0$, $M(Y) \succ 0$ for every Y if and only if

$$m_\Sigma J_{\text{eff}} > \tfrac{L_b^2}{4}\, m_\Delta^2 \quad (9)$$

(Appendix A.3). To ensure that ~~well-posedness and~~[own] (9) ~~hold during training~~ holds for every estimate of θ_{base}[own], we use a logarithmic parametrisation for the nine positive combinations and a tanh-bounded m_Δ ~~(Section IV-C)~~[M-mail]. [from IV-C, M-mail] ~~The optimiser updates~~ We estimate[own] θ_{base} ~~through training~~[own] coordinates $\beta \in \mathbb{R}^{10}$, which are zero at the ~~initial parameters θ^0_{base}~~ initial estimate θ^0_{base}[own] ~~and ensure that (9) holds at every update~~[own]. The nine positive parameter combinations ~~j of (8)~~[own] are $\theta_{\text{base},j} = \theta^0_{\text{base},j} \exp(\beta_j)$for each positive $\theta_{\text{base},j}$ in (8)[own], while m_Δ can be positive or negative and is bounded by (9):

$$m_\Delta = \gamma\, \frac{2}{L_b} \sqrt{m_\Sigma J_{\text{eff}}}\, \tanh\big(\eta_0 + s\, \beta_{m_\Delta}\big), \quad (10)$$

where ~~β_Δ is the entry of β for m_Δ~~ β_{m_Δ} is the coordinate of m_Δ[own] and $\gamma \in (0, 1)$ provides a safety margin. We choose the offset η_0 and scale s so that $\beta = 0$ gives $m_\Delta = m_{\Delta,0}$ ~~and $\partial m_\Delta / \partial \beta_\Delta = m_{\Delta,0}$. This matches the logarithmic parameters, where $\partial \theta_{\text{base},j} / \partial \beta_j = \theta^0_{\text{base},j}$ at $\beta = 0$~~[own] (Appendix A.3). ~~The initial value must satisfy $|m_{\Delta,0}| < \gamma(2/L_b)\sqrt{m_{\Sigma,0} J_{\text{eff},0}}$. The bound in (9) holds for every β, therefore no clamping or penalty is needed.~~[own end]

IV. Dynamic Augmentation of the Physics Baseline

- The section extends the baseline with learned dynamics and identifies the result from closed-loop records; it names its five subsections (style profile)
- Contribution share: the realisation on the self-scheduled gantry baseline with joint estimation of θ_{base}, its identification in closed loop with the known controller, and the PS initialisation of the added states (MUST ESTABLISH)

[Changes: black = original, blue = original, moved here (from … end), green = changed or new, ~~struck~~ = deleted ("moved to" = moved, not deleted), superscript = comment id.]

~~The dynamics of the system that are not captured by the baseline model defined in Section III-B will be identified from input-output data measured in closed-loop. The baseline model will be augmented to represent these missing dynamics.~~[moved to II, Jan7] ~~A correction to the states of the baseline model alone is not sufficient, because missing dynamics need states the baseline does not have (Section IV-A), so additional states are introduced. We make use of a feedforward neural network to model both the changes for the baseline states and the additional states.~~[moved to II, Jan7] In this section, the baseline model of Section III is augmented to represent the missing dynamics introduced in Section II. We define the augmented model ~~(Section IV-A)~~[Dirk], simulate it inside the known feedback loop ~~(Section IV-B)~~[Dirk], estimate it jointly with θ_{base} ~~(Section IV-C)~~[Dirk], and start the training from the baseline ~~(Section IV-D)~~[Dirk] with a data-driven initialisation for the added states ~~(Section IV-E)~~[Dirk].

A. Augmented model

- If the omitted effects are dynamic, the six baseline states cannot represent them; refitting θ_{base} or a static augmentation adds no states (EJC Sec. 2)
- Hoekstra's LFR augmentation structure with a fixed interconnection; the self-scheduled LPV-LFR is its baseline block; the learned block is a network, not Drenth's LPV-LFR augmentation (thesis Eq. (5.2)) (?: reason)
- Parallel because the orthogonal-by-construction method of Section IV is defined for the additive structure (Györök 2026); output map not augmented (Kessels Remark 5.3)
- The structure is a discrete-time state update (EJC Sec. 2, Table 1), so f_{base} is one RK4 step with the input held and $M(Y)$ per stage; the correction is added after the step
- Acyclic interconnection plus the baseline result of Section II: well posed for every Y (EJC Thm. 1)
- Normalisation: affine, statistics of the training records, velocities from differenced positions; later displays in normalised symbols
- One tanh MLP supplies f_{aug} and g_{aug} (EJC Sec. 4.3); it corrects every physical row, so a persistent correction on X or Y accumulates in position (?: justification); $n_{\bar{x}} = 0$ is the static parallel augmentation
- The baseline/network split is not unique (2026 Sec. 5.2); interpretable means the estimated θ_{base} keeps its physical meaning (Györök Sec. 3); Section IV makes the estimate unique

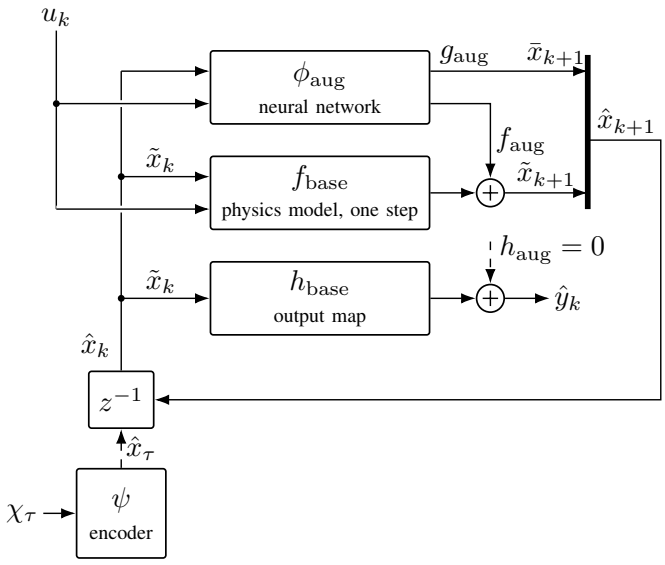


Fig. 3: Augmented model. The network ϕ_{aug} is connected in parallel with the baseline state-transition function f_{base}, which is one RK4 step of the physics model over T_s. Both f_{base} and h_{base} use the physical states $\tilde{x}_k$ of $\hat{x}_k$. The encoder ψ estimates the initial state $\hat{x}_\tau$ for each simulation window. [**TODO:** No setpoint generator currently for the inputs; layout cramped at one column width.]

~~To represent additional dynamics such as a flexible mode, refitting θ_{base} or correcting the baseline states with a static augmentation, where the learned correction is only added to the physical states, is not sufficient. Both these methods only retain the original states of the baseline~~[own], ~~while missing dynamics require states the baseline does not include [?]Sec. 2]hoekstra2026lfr.~~[own] A dynamic augmentation ~~therefore~~[Dirk] adds states $\bar{x}$ to model ~~these~~ the[Dirk] missing dynamics, which require states the baseline does not include (Section I)[Dirk]. The augmented model updates these states using ~~the full state and input~~ the baseline states, the added states and the actuator forces[own] [5, Sec. 2, Table 1].

We use the dynamic parallel structure of Hoekstra et al. [5, Table 1][own] ~~(Fig. 3)[Dirk], here a neural network adds it correction on the same state as the baseline model in parallel~~[own]. ~~besides the baseline model on the same state.~~[own] [from IV intro, Dirk] We make use of a feedforward neural network to model both the changes for the baseline states and the additional states.[end] Fig. 3 shows how the network is connected in parallel with the baseline model.[M-p6] ~~The learning components are a neural network consisting of f_{aug} and g_{aug}.~~[own] The model is defined in discrete time with the augmentation added to the state update of the baseline ~~[?]Sec. 2, Table 1]hoekstra2026lfr~~[own]. This gives us

$$\begin{aligned}\tilde{x}_{k+1} &= f_{\text{base}}(\theta_{\text{base}}, \tilde{x}_k, u_{\text{act},k}) + f_{\text{aug}}(\theta_{\text{aug}}, \hat{x}_k, u_{\text{act},k}),\\ \bar{x}_{k+1} &= g_{\text{aug}}(\theta_{\text{aug}}, \hat{x}_k, u_{\text{act},k}),\\ \hat{y}_k &= P^\top q_k,\end{aligned} \tag{11}$$

where $\tilde{x}_k = [q_k^\top \ \dot{q}_k^\top]^\top \in \mathbb{R}^6$ is the physical state of ~~(43)~~ (7)[Dirk]in generalised coordinates[own], $\bar{x}_k \in \mathbb{R}^{n_{\bar{x}}}$ the added state, the augmented state $\hat{x}_k = [\tilde{x}_k^\top \ \bar{x}_k^\top]^\top$, ~~$f_{\text{base}}$ the baseline state update~~ f_{base} and $h_{\text{base}}(\tilde{x}_k) = P^\top q_k$ the baseline state-transition and output functions[own], $u_{\text{act},k} \in \mathbb{R}^3$ the actuator forces, $\hat{y}_k \in \mathbb{R}^3$ the predicted actuator positions, ~~and y_k the measured positions of (1),~~[own] $\theta_{\text{base}} \in \mathbb{R}^{10}$ the combinations (8), and [**TODO:** using theta might be confusing with baseline] $\theta_{\text{aug}} \in \mathbb{R}^{n_{\text{aug}}}$ the network parameters. The added states ~~are updated by g_{aug}~~[own]have[own] ~~without any~~ no[own] predetermined physical meaning, in contrast to the baseline states $\tilde{x}$ ~~for which the positions are measured~~ , which are positions and velocities[own].

The dynamic-parallel augmentation structure is chosen, because the orthogonal-by-construction ~~parametrisation that Section V builds upon~~ augmentation[M-mail] is defined for this additive structure [8].

The output map is not augmented ($h_{\text{aug}} = 0$ in Fig. 3), because (3) gives the exact relation between the states and the measured outputs~~, as in Kessels [?]Remark 5.3]kessels2025ai[own]. This is also done to~~ , and this ensures[own] ~~ensure~~ [own]that q still represents the physical positions. ~~With a learned correction to the output map the states could stop representing the actual positions.~~[own] ~~The baseline is evaluated over one sample period T_s.~~[own] ~~Writing (4) in state-space form gives the state derivative F, where $u = Pu_{\text{act}}$ is the generalised force (3),~~[moved to III-B, Dirk] ~~The baseline state derivative F is given by (7), where M, C and K depend on θ_{base} through (8)[own], and the expression for $M(Y)^{-1}$ follows from analytically resolving the baseline LFR loop (Appendix A.2)~~[Dirk]~~.~~[own] The baseline state-transition function [own]f_{base}(Fig. 3)[M-p6] is calculated using one fourth-order Runge-Kutta (RK4) step of F(7)[own] over T_s, with the input $u = Pu_{\text{act}}$ (3)[own] held constant (zero-order hold) during the step. For each of the four RK4 sub-steps $M(Y)$ is evaluated using the value of Y from its intermediate state.

[**TODO:** The augmented model admits a unique solution for every state, input and Y. The interconnection has no algebraic loops, making it acyclic, and both blocks are twice continuously differentiable, which together give well-posedness [5, Cond. 6, Cond. 8, Thm. 9]. The baseline LFR is also well posed (Section III-B).]

from IV-A, after normalisation, own ~~A multilayer perceptron ϕ_{aug} with tanh activation functions for the hidden layers and a linear output layer is used for both f_{aug} and g_{aug}.~~ The feedforward neural network ϕ_{aug} is a multilayer perceptron with tanh activation functions for the hidden layers and a linear output layer.[own] The inputs to the neural network are the six baseline states$\tilde{x}_k$[M-p5],[M-p5] ~~and~~[M-p5] $n_{\bar{x}}$ added states$\bar{x}_k$[M-p5] and the $n_u = 3$ actuator forces$u_{\text{act},k}$[M-p5]. The first six outputs ~~$f_{\text{aug}}^{\text{n}}$ update~~ give the corrections of[own] the six physical states and the remaining $n_{\bar{x}}$ outputs ~~$g_{\text{aug}}^{\text{n}}$~~ g_{aug}[own] equal the additional states $\bar{x}_{k+1}$ (Fig. 3). ~~These corrections act on all six physical states, the positions included, as in the dynamic-parallel structure [?]Table 1]hoekstra2026lfr.~~[own] The positions are also correctedas in the dynamic parallel structure [5, Table 1][own],[own] because within one sample period an unmodelled force would change both the velocity and the [own]position~~, correcting only for the velocities would not be able to capture this~~[own]. ~~Both f_{aug} and g_{aug} are given the payload position Y through $\tilde{x}_k$, enabling~~ The payload position Y in $\tilde{x}_k$ enables[own] the neural network to learn position-dependent effects.[end]

The inputs to the neural network are normalised to zero mean and unit variance [19], as in [5, Sec. 5.3][Q41]. [**TODO:** Add bib entry `bishop1995neural`: C. M. Bishop, *Neural Networks for Pattern Recognition*, Oxford University Press, 1995 (the source Hoekstra et al. cite for this claim, their ref. [6], Sec. 5.3).] The ~~mean~~ means μ[own] and standard deviationsσ[own] are computed from ~~the training data~~ $\mathcal{D}_N$[own]. ~~For the baseline states the measured positions are converted to their generalised positions,~~ The baseline states are in generalised coordinates, so the measured positions are converted to $q = P^{-\top} y$,[own] for which the velocities can be estimated from their finite difference by dividing with T_s. This ensures that the positions and outputs are scaled with the same measurements. ~~This also makes sure that all models independent of its initial parameters use the same scale,~~[own] The normalisation is[own]

$$\tilde{x}^{\text{n}} = T_x(\tilde{x} - \mu_x), \quad u^{\text{n}} = T_u(u_{\text{act}} - \mu_u), \quad y^{\text{n}} = T_y(y - \mu_y), \tag{12}$$

where $\mu_x \in \mathbb{R}^6$ and $\mu_u, \mu_y \in \mathbb{R}^3$ are the ~~channel means~~ sample means of $\tilde{x}$, u_{act} and y[own], and $T_i = \text{diag}(\sigma_i)^{-1}$ for $i \in \{x, u, y\}$, with σ_i the corresponding standard deviations.[from after (13), M-p6] RK4 evaluates F with the states and inputs in SI units, which ~~where $\tilde{x}$ and u_{act}~~[M-p6] follow from $\tilde{x}^{\text{n}}$ and u^{n} by inverting (12)~~, so RK4 uses the states and inputs in SI units~~[M-p6].[end] ~~The additional states $\bar{x}$ are not normalised separately, as their initial scale is provided by the neural-network initialisation and the normalised input data,~~ We normalise only the baseline states, since the added states $\bar{x}$ have no physical units. The scale of $\bar{x}$ is set by the network initialisation and its normalised inputs.[Dirk] ~~so~~ The augmented state is then[Dirk] $\hat{x}^{\text{n}} = [\tilde{x}^{\text{n}\top} \ \bar{x}^\top]^\top$. [**TODO:** The additional states $\bar{x}$ are not normalised separately. Their scale is learned by the encoder and the added-state transition network, operating on normalised physical signals.] After normalisation the model equations are

$$\begin{aligned}
\tilde{x}^{\text{n}}_{k+1} &= f^{\text{n}}_{\text{base}}(\theta_{\text{base}}, \tilde{x}^{\text{n}}_k, u^{\text{n}}_k) \\
&\quad + f^{\text{n}}_{\text{aug}}(\theta_{\text{aug}}, \hat{x}^{\text{n}}_k, u^{\text{n}}_k), \\
\bar{x}_{k+1} &= g^{\text{n}}_{\text{aug}}(\theta_{\text{aug}}, \hat{x}^{\text{n}}_k, u^{\text{n}}_k), \\
\hat{y}^{\text{n}}_k &= C^{\text{n}} \tilde{x}^{\text{n}}_k, \qquad C^{\text{n}} = T_y P^\top [I_3 \ \ 0]\, T_x^{-1}, \\
f^{\text{n}}_{\text{base}}(\theta_{\text{base}}, \tilde{x}^{\text{n}}, u^{\text{n}}) &= T_x\big(f_{\text{base}}(\theta_{\text{base}}, \tilde{x}, u_{\text{act}}) - \mu_x\big), \\
\tilde{x} &= T_x^{-1}\tilde{x}^{\text{n}} + \mu_x, \qquad u_{\text{act}} = T_u^{-1} u^{\text{n}} + \mu_u.
\end{aligned} \tag{13}$$

[**Note:** (17) rewritten (Jan33, Maarten p6): rows 1 to 3 are (14) in normalised coordinates, rows 4 and 5 define the normalised baseline with the mapping back. Same math as before. Old versions in the source comment. Remove this note after checking.] ~~where $\tilde{x}$ and u_{act} follow from $\tilde{x}^{\text{n}}$ and u^{n} by inverting (12), so RK4 uses the states and inputs in SI units.~~[moved to after (12), M-p6] ~~Because f_{aug} is added to f_{base}, whose mean μ_x is already subtracted in $f^{\text{n}}_{\text{base}}$, the correction is only scaled by T_x.~~ We scale f_{aug} only by T_x, since $f^{\text{n}}_{\text{base}}$ already

subtracts μ_x.[Dirk] [**TODO:** The correction changes the predicted state, so its normalised form is $f^{\mathrm{n}}_{\mathrm{aug}} = T_x f_{\mathrm{aug}}$, without subtracting μ_x. In the implementation, the network directly outputs $f^{\mathrm{n}}_{\mathrm{aug}}$.] ~~The output map remains linear without offset because the mean of the positions in μ_x is calculated from the output mean as $P^{-\top}\mu_y$, so that μ_y cancels in $T_y(P^\top q - \mu_y)$.~~ We set the position means in μ_x to $P^{-\top}\mu_y$, so μ_y cancels in $T_y(P^\top q - \mu_y)$ and the output map remains linear without an offset.[Dirk] We include the means in $f^{\mathrm{n}}_{\mathrm{base}}$, whereas the transformation in [5, Sec. 5.3] assumes zero-mean signals.[Dirk] ~~A multilayer perceptron ϕ_{aug} with tanh activation functions for the hidden layers and a linear output layer is used for both $f^{\mathrm{n}}_{\mathrm{aug}}$ and $g^{\mathrm{n}}_{\mathrm{aug}}$.~~[moved to IV-A, before normalisation, own] The network ϕ_{aug} uses the normalised signals:[Dirk]

$$\left[f^{\mathrm{n}\top}_{\mathrm{aug}} \;\; g^{\mathrm{n}\top}_{\mathrm{aug}}\right]^\top = \phi_{\mathrm{aug}}(\theta_{\mathrm{aug}}, \hat{x}^{\mathrm{n}}_k, u^{\mathrm{n}}_k).$$

~~The inputs to the neural network are the six baseline states and n_x added states and the $n_u = 3$ actuator forces. The first six outputs $f^{\mathrm{n}}_{\mathrm{aug}}$ update the six physical states and the remaining n_x outputs $g^{\mathrm{n}}_{\mathrm{aug}}$ equal the additional states $\tilde{x}_{k+1}$ (Fig. 3). These corrections act on all six physical states, the positions included, as in the dynamic-parallel structure [?]Table 1]hoekstra2026lfr. The positions are also corrected because within one sample period an unmodelled force would change both the velocity and position, correcting only for the velocities would not be able to capture this. Both f_{aug} and g_{aug} are given the payload position Y through $\tilde{x}_k$, enabling the neural network to learn position-dependent effects.~~[moved to IV-A, before normalisation, own] ~~The gantry baseline model has two axes, X and Y, without stiffness to ground in (5), which are also affected by the learned correction. The linearised RK4 step has a pole at $z = 1$ on each of these axes, making it marginally stable: stable, but not asymptotically stable. Therefore these axes act as integrators: a constant error in the correction keeps accumulating in the position. The alternative to only correct the velocities would not prevent the integrator issue, because an error in the velocity also accumulates in the position. The feedback controller is able to correct this drift, which is why we have chosen to simulate the model in closed-loop (Section IV-B).~~[moved to IV-B, M-p6]

B. Closed-loop simulation

- The model is not driven by the recorded input (2026 Eq. (22)), for two reasons: noise-induced bias under feedback (Kessels Remark 5.6, SUBNET; indirect approach, Forssell and Ljung) and the free axes of Section IV-A, which violate the stability condition behind SUBNET consistency (Beintema Cond. 1); motivation, not a proof
- As in Kessels (Eqs. (5.12), (5.13)), each window is simulated in closed loop with the known controller, which forms its error against the model output; system and model loops share r and u^{ff}
- The controller is linear and time invariant, so subtracting the system's loop cancels r and u^{ff}; the residual form needs only the recorded u_{act} and y, and is exact when u_{act} was produced by this controller from y (this work)
- $x^c_\tau = 0$ starts the model's controller in the system's controller state, as the encoder does for the plant state; no controller state is estimated, unlike Kessels Remark 5.4 (this work)
- No plant feedthrough fixes the step order, so D_{fb} closes no algebraic loop (as Kessels Remark 5.1); one normalised step displayed
- The output error re-enters the model input, so part of it is corrected within the window; error before τ is not carried in, the free-run validation carries it

~~The data is measured in closed-loop, and the model is simulated in closed-loop.~~[own] [from IV-B, below, Dirk] ~~Every training window is~~ The model is[M-p7] simulated in closed-loop with ~~the assumed known controller~~ the controller K_{fb} of (2)[own], as in Kessels [4, Eqs. (5.12), (5.13)][end], instead of using the recorded input as in [5, Eq. (22)][Dirk]. ~~Instead of using the recorded input as in [?]Eq. (22)]hoekstra2026lfr, we include the feedback controller in the simulation[Dirk] for two reasons[Jan49].[Dirk]~~ The reason is that the open-loop model drifts.[own] ~~First, as established in Section IV-A: the axes X and Y without stiffness make the open-loop gantry marginally stable.~~[M-p6] [from IV-A, M-p6] The gantry baseline model has two axes, X and Y, without stiffness to ground in (5), which are also affected by the learned correction. The linearised RK4 step has a pole at $z = 1$ on each of these axes, ~~making it marginally stable: stable, but~~ so it is[own] not asymptotically stable. Therefore these axes act as integrators: a constant error in the correction keeps accumulating in the position. The alternative to only correct the velocities would not prevent the integrator issue, because an error in the velocity also accumulates in the position. When the model is driven by the recorded input, nothing corrects this drift.[own] The feedback controller is able to correct this drift~~, which is why we have chosen to simulate the model in closed-loop (Section IV-B).~~[M-p6 end] ~~SUBNET, the encoder-based identification method of Beintema et al. [15], assumes that the system that generates the data is incrementally exponentially output stable [15]. The identification method we have adapted from [5] relies on this SUBNET assumption [5]. The open-loop gantry system does not satisfy this assumption: a mismatch in the initial velocity will create a permanent position offset. In closed-loop the feedback controller is able to correct this offset.~~[moved to IV-C, M-p6]

~~Second, the feedback controller causes a correlation between the recorded input and the measurement noise. Kessels was not able to identify an open-loop SUBNET model using closed-loop data from a wire bonder. Kessels cites earlier work suggesting that this could be explained by noise-induced bias due to this correlation and the integral term of the controller [4].[Jan49] This bias is avoided in the indirect approach for linear systems with a known controller, which fits the closed-loop model using the reference. Because the reference is uncorrelated with the noise, the model can be identified from the output error [20].[M-p6] This motivates the choice for simulating in closed-loop.[own] They do not prove that our estimate is consistent.[Jan49]~~ [**TODO:** Missing: evidence on the thesis data that open-loop rollouts accumulate

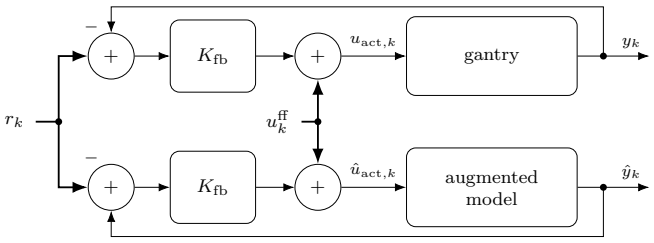


Fig. 4: Closed-loop system (top) and closed-loop model (bottom). Both use the same controller K_{fb}, reference r_k and feedforward u_k^{ff}. In the model, the controller uses the model output $\hat{y}_k$.

position error on X and Y. Candidate: the open-loop versus closed-loop drift on one record that `THESIS-RESULTS.md` Sec. 5 plans, reported in Section VII.]

~~The model is simulated in closed-loop with the controller K_{fb} of (2), as in Kessels [?]Eqs. (5.12), (5.13)]kessels2025ai.~~[moved to IV-B start, Dirk] ~~The feedback controller is linear and time invariant, the model output is used to calculate the position error in actuator coordinates. The controller calculates the actuator forces from this error.~~[own] The system and model use the same reference r and feedforward u^{ff}, but use their own output for feedback, with controller states x^{fb} and $\hat{x}^{\mathrm{fb}}$ (Fig. 4)~~, with the system loop of (2)~~[M-p3]. The closed-loop system is given by (1) and (2). In the closed-loop model, the controller uses the model output $\hat{y}_k$:[own] ~~[system rows of this equation moved to (2)]~~[moved to II, M-p3]

$$\begin{aligned} \hat{x}^{\mathrm{fb}}_{k+1} &= A_{\mathrm{fb}}\hat{x}^{\mathrm{fb}}_k + B_{\mathrm{fb}}(r_k - \hat{y}_k), \\ \hat{u}_{\mathrm{act},k} &= C_{\mathrm{fb}}\hat{x}^{\mathrm{fb}}_k + D_{\mathrm{fb}}(r_k - \hat{y}_k) + u^{\mathrm{ff}}_k, \end{aligned} \tag{14}$$

~~where $(A_{\mathrm{fb}}, B_{\mathrm{fb}}, C_{\mathrm{fb}}, D_{\mathrm{fb}})$ are the state-space matrices of the controller discretised with sampling time T_s. Both r_k and u_k^{ff} are in actuator coordinates.~~[moved to II, M-p3] The model uses the calculated actuator force $\hat{u}_{\mathrm{act},k}$ instead of the recorded $u_{\mathrm{act},k}$ in (11).

Since we use a linear controller we are able to subtract ~~the system and model controller equations~~ (2) from (14)[own] to cancel r and u^{ff}. This leaves us with the controller state difference $x^c = \hat{x}^{\mathrm{fb}} - x^{\mathrm{fb}}$, driven by $y - \hat{y}$. The model is still simulated with respect to the reference r, but r and u^{ff} are not used in simulation. ~~If the controllers between the system and simulation differ, the equations are only approximate and noise-induced bias cannot be ruled out.~~[moved to below (15), own]

At the start of ~~each training window~~ a simulation at sample τ[M-p7] we set $x^c_\tau = 0$~~, which sets~~ . This sets[own] the model's controller state equal to that of the system at τ, because x^c is the difference between the two controller states[own]. ~~Therefore the encoder only has to initialise the states of the model. Kessels instead needs the system's controller state at the start of each window, either measured directly or calculated by simulating the controller from the start of the record [4].~~[moved to IV-C, M-p7] [from IV-C, M-p7] Therefore the encoderof Section II[own] only has to initialise the states of the model. Kessels instead needs the system's controller state at the start of each window, either measured directly or calculated by simulating the controller from the start of the record [4, Remark 5.4].[end] We then write the closed-loop model in (14) as [**Note:** ZOH eigenlijk beter dan tustin. en tustin is voor een frequency domein het beste. dus tustin niet de beste keuze. Jasper denkt dat ZOH de juiste keuze is ipv tustin voor controller discretizeren.] [**Note:** als je discretizeert om tijd te besparne, dat overal doen. checken of validatie in 4 of 20khz is. met butterworth gooi de data weer weg, dus validatie op 20kzh onlogisch.] [**Note:** verwarrend voor de lezer als je een discrete controller hebt maar in continuous time laat zien.]

$$\begin{aligned} x^c_{k+1} &= A_{\mathrm{fb}}x^c_k + B_{\mathrm{fb}}(y_k - \hat{y}_k), \qquad x^c_\tau = 0, \\ \hat{u}_{\mathrm{act},k} &= u_{\mathrm{act},k} + C_{\mathrm{fb}}x^c_k + D_{\mathrm{fb}}(y_k - \hat{y}_k), \end{aligned} \tag{15}$$

which only needs the recorded u_{act} and y. [**TODO:** Should be discussed in the setup for controller discretisation] [**TODO:** Check for Setup: the training loop uses the controller re-discretised at the model rate, while the records were generated at a higher rate. Run the re-discretised controller on the recorded $r - y$ and compare its feedback force with the recorded one. A rollout with $\hat{y} = y$ gives $\hat{u} = u$ by construction and checks nothing. Related: `FINAL-RUN-FIXES.md` item 3, the D-232 leak check on the lowpass data, passed (0.22 to 0.50 nm).] [from App. C.3, own] Let ~~$\bar{u}_{\mathrm{act},k}$~~ $\check{u}_{\mathrm{act},k}$[own] be the force that K_{fb} computes from the recorded r_k and y_k, with state ~~$\bar{x}^{\mathrm{fb}}_k$~~ $\check{x}^{\mathrm{fb}}_k$[own], plus u^{ff}_k. With ~~$\hat{x}^{\mathrm{fb}}_k = \bar{x}^{\mathrm{fb}}_k + x^c_k$~~ $\hat{x}^{\mathrm{fb}}_k = \check{x}^{\mathrm{fb}}_k + x^c_k$[own], (15) gives

$$\hat{u}_{\mathrm{act},k} = C_{\mathrm{fb}}\hat{x}^{\mathrm{fb}}_k + D_{\mathrm{fb}}(r_k - \hat{y}_k) + u^{\mathrm{ff}}_k + \big(u_{\mathrm{act},k} - \check{u}_{\mathrm{act},k}\big). \tag{16}$$

The first three terms are the ~~model loop~~ closed-loop model[own]. The last term is ~~the force error of K_1 at T_s above~~ the difference between the forces from the system and model controllers[Dirk] [end] [from above, own] The noise v is independent of r and u^{ff} (Section II).[own] If the controllers between the system and simulation differ, the equations are only approximate ~~and noise-induced bias cannot be ruled out~~ and the noise enters through this last term[own]. The noise also enters through the initial state at τ. The encoder computes it from the recorded u_{act} and y, and the controller state at τ depends on earlier measurements y.[own] [end]

The model output$\hat{y}_k$ depends only on the state$\tilde{x}_k$, not on the input ~~$\hat{u}_k$~~ $\hat{u}_{\mathrm{act},k}$[own]at the same sample[Is98]. For each sample we first calculate the output~~ and then~~ , then[own] the error and the [own]model input,[own] before updating the states. This avoids an algebraic loop through D_{fb}, for the same reason as Kessels [4, Remark 5.1]. For a ~~window~~ simulation[M-p7] starting at τ, the model equations in the normalised coordinates of (12) are

$$\begin{aligned} \hat{y}^{\mathrm{n}}_{k|\tau} &= C^{\mathrm{n}}\tilde{x}^{\mathrm{n}}_{k|\tau}, \qquad \varepsilon_{k|\tau} = T_y^{-1}\big(y^{\mathrm{n}}_k - \hat{y}^{\mathrm{n}}_{k|\tau}\big), \\ \hat{u}^{\mathrm{n}}_{k|\tau} &= u^{\mathrm{n}}_k + T_u\big(C_{\mathrm{fb}}x^c_k + D_{\mathrm{fb}}\varepsilon_{k|\tau}\big), \\ x^c_{k+1} &= A_{\mathrm{fb}}x^c_k + B_{\mathrm{fb}}\varepsilon_{k|\tau}, \qquad x^c_\tau = 0, \end{aligned} \tag{17}$$

and the next model state $\hat{x}^{\mathrm{n}}_{k+1|\tau}$ is calculated using (13) with input $\hat{u}^{\mathrm{n}}_{k|\tau}$. ~~Each window starts from the encoder state in (19).~~[M-p8] ~~The controller uses the output error to correct the model input within the training window, as in the closed loop system. Each window starts from a new encoder estimate and $x^c_\tau = 0$, so the window is not affected by model errors before~~

~~τ. The validation of Section IV-C is simulated in closed-loop and only initialised at the start.~~[moved to IV-C, M-p8]

C. Identification

- θ_{base} is estimated jointly with network and encoder, as in 2026 Eq. (22); Kessels trains only the network weights (Eq. (5.12))
- Training coordinates β, zero at θ^0_{base}: nine combinations logarithmic, m_Δ through (10); every β admissible, so no clamp or penalty (this work, D-229; adapted from Drenth Sec. 4.2)
- Criterion (18): Eq. (22) adapted by the loop of Section IV-B and the encoder of Section IV-D; every window sample scored; controller fixed; a closed-loop fit can mask plant error (Section VI)
- The baseline/network split is not unique (2026 Sec. 5.2), so θ_{base} is not interpretable in the sense of Section I; Section IV addresses this (unique to first order only?)
- Adam, then an unbounded L-BFGS polish (adapted from Drenth Sec. 4.1); checkpoint with the lowest closed-loop free-run error over whole validation records; polish kept only if it lowers that error
- Initialisation (23): $\tilde{x}$ and $\hat{y}$ reproduce the baseline, $\bar{x}$ evolves through a live g_{aug} (2026 Eq. (29)); adapted from Sec. 5.4.3 (random added-state rows, no linear bypass); the model is named in Section VI, not here

This subsection states how the augmented model is estimated from ~~the records: which parameters are trained, the criterion, the encoder that gives each simulation window its initial state, and the optimiser~~ $\mathcal{D}_N$. We first motivate the encoder, then give the cost function, the encoder and the optimiser[own]. ~~The model is trained using short windows of the closed-loop data, where each window is simulated with the known controller. The initial states are estimated by an encoder network, including the additional states.~~[moved to II, M-p7] [from II, Jan23] ~~During training, the physical parameters θ_{base} are also estimated, using the combinations defined in Section III-C.~~[own end] ~~This is the SUBNET approach of Beintema et al. [15]: the simulation error over short windows is minimised, and an encoder, trained jointly with the model, estimates the initial state of each window from past inputs and outputs.~~[moved to II, M-p7] ~~The network parameters θ_{aug}, the encoder parameters θ_{enc} and θ_{base} are jointly estimated, as in [5], while the encoder paper keeps θ_{base} fixed [21].~~[moved to IV-C, below, M-p8] ~~The optimiser updates θ_{base} through training coordinates $\beta \in \mathbb{R}^{10}$, which are zero at the initial parameters θ^0_{base} and keep every iterate admissible (Section III-C). The nine positive parameter combinations j of (8) are $\theta_{\text{base},j} = \theta^0_{\text{base},j} \exp(\beta_j)$, while m_Δ can be positive or negative and is bounded by (9): [equation (10) moved to Section III-C] where β_Δ is the entry of β for m_Δ and $\gamma \in (0,1)$ provides a safety margin. We choose the offset η_0 and scale s so that $\beta = 0$ gives $m_\Delta = m_{\Delta,0}$ and $\partial m_\Delta / \partial \beta_\Delta = m_{\Delta,0}$. This matches the logarithmic parameters, where $\partial \theta_{\text{base},j} / \partial \beta_j = \theta^0_{\text{base},j}$ at $\beta = 0$ (Appendix A.3). The initial value must satisfy $|m_{\Delta,0}| < \gamma(2/L_b)\sqrt{m_{\Sigma,0} J_{\text{eff},0}}$. The bound in (9) holds for every β, therefore no clamping or penalty is needed.~~[moved to III-C, M-mail] ~~The parameters $\vartheta = (\beta, \theta_{\text{aug}}, \theta_{\text{enc}})$ are trained to minimise the normalised output error of the closed-loop simulation, with the controller fixed. We adapt this from [5], adapted for the closed loop of Section IV-B and the encoder described below: [criterion (18) moved below] for $\tau \le k < \tau + n_f$, where each training window starts at a sample $\tau \in \mathcal{W}$ and has length n_f, $n_y = 3$, and $\theta_{\text{base}}(\beta)$ gives the baseline parameters. $W_r = \text{diag}(\sigma_{y,j}/\sigma_{e,r,j})$ weights axis j of training record r, so that its residual in metres is divided by the standard deviation $\sigma_{e,r,j}$ of its servo error (38).~~[moved to IV-C, below, M-p8]

The data $\mathcal{D}_N$ consist of multiple records.[own] Each simulation window startsat a sample τ from an initial state $\hat{x}_\tau$, ~~and k denotes the sample within the window~~[own], but we measure only the positions of the physical state $\tilde{x}$. The additional states $\bar{x}$ are latent, but we need their initial values at τ to simulate each window. Within the window, k denotes the sample.[own] Estimating $\hat{x}_\tau$ as a separate parameter for each window would increase the number of parameters as the data grows [15, Sec. 3.1]. Following Beintema et al., we use an encoder network that estimates $\hat{x}_\tau$ from the [own]past inputs and outputsχ_τ[own] [15, Sec. 3.2]. This keeps the computation per gradient step independent of the record length,[own] for a [own]fixed batch size and window ~~lengths~~ length[own] [15, Sec. 3.1, Eq. (9)]. Using short simulation windows increases the loss function smoothness [15, Sec. 3.1, 4.2].

[from IV-C start, M-p8] The network parameters θ_{aug}, the encoder parameters θ_{enc} and θ_{base} are jointly estimated, as in [5, Sec. 5.1], while the encoder paper keeps θ_{base} fixed [21, footnote 1].[end] [from IV-C start, M-p8] The parameters $\vartheta = (\beta, \theta_{\text{aug}}, \theta_{\text{enc}})$ are trained to minimise the normalised output error of the closed-loop simulation, with the ~~controller fixed~~ known controller K_{fb}[own]. We adapt this from [5, Eq. (22)], ~~adapted~~[own] for the closed loop of Section IV-B and the encoder ~~described below~~[M-mail]:

$$\min_{\vartheta} \; V(\vartheta) = \frac{1}{|\mathcal{W}|\, n_f\, n_y} \sum_{\tau \in \mathcal{W}} \sum_{k=\tau}^{\tau+n_f-1} \left\| W_r \left(y^{\text{n}}_k - \hat{y}^{\text{n}}_{k|\tau} \right) \right\|_2^2 \quad \text{s.t.}$$
$$\tilde{x}^{\text{n}}_{k+1|\tau} = f^{\text{n}}_{\text{base}}\left(\theta_{\text{base}}(\beta), \tilde{x}^{\text{n}}_{k|\tau}, \hat{u}^{\text{n}}_{k|\tau}\right) + f^{\text{n}}_{\text{aug}}\left(\theta_{\text{aug}}, \hat{x}^{\text{n}}_{k|\tau}, \hat{u}^{\text{n}}_{k|\tau}\right),$$
$$\bar{x}_{k+1|\tau} = g^{\text{n}}_{\text{aug}}\left(\theta_{\text{aug}}, \hat{x}^{\text{n}}_{k|\tau}, \hat{u}^{\text{n}}_{k|\tau}\right),$$
$$\hat{x}^{\text{n}}_{\tau|\tau} = \psi(\theta_{\text{enc}}, \chi_\tau), \quad \hat{y}^{\text{n}}_{k|\tau},\ \hat{u}^{\text{n}}_{k|\tau} \text{ from (17)}, \tag{18}$$

for $\tau \le k < \tau + n_f$, where each training window starts at a sample $\tau \in \mathcal{W}$of record r[own] and has length n_f, $n_y = 3$, and $\theta_{\text{base}}(\beta)$ gives the baseline parameters(Section III-C)[own]. $W_r = \text{diag}(\sigma_{y,j}/\sigma_{e,r,j})$ weights axis j of ~~training record r~~ record r[own], so that its residual in metres is divided by the standard deviation $\sigma_{e,r,j}$ of its servo error ~~(38)~~ $y_j - r_j$[M-mail]. [**TODO:** might have to move to discussion if i dont add it. Since the controller corrects part of the output error, it can hide errors in the plant model. We therefore also test the model with a different controller in Section VII.][end]

~~The encoder uses~~ In (18), the encoder ψ uses[M-p8] the samples

up to $\tau-1$, as in [5, Eq. (22e)]:

$$\begin{aligned}\hat{x}^{\mathrm{n}}_{\tau|\tau} &= \psi(\theta_{\mathrm{enc}}, \chi_\tau),\\ \chi_\tau &= \big[\,(y^{\mathrm{n}}_{\tau-n-1})^\top \;\cdots\; (y^{\mathrm{n}}_{\tau-1})^\top \\ &\qquad (u^{\mathrm{n}}_{\tau-n-1})^\top \;\cdots\; (u^{\mathrm{n}}_{\tau-1})^\top\big]^\top,\end{aligned} \tag{19}$$

where $\hat{x}^{\mathrm{n}}_{\tau|\tau}$ is the normalised state $\hat{x}_\tau$ of Fig. 3 at the start of the window, $n+1$ is the number of past [own]samples ~~in the window~~ used by the encoder[own], and [own]$\chi_\tau \in \mathbb{R}^{6(n+1)}$ stacks the normalised recorded outputs and inputs as in (12)~~, and $\theta_{\mathrm{enc}} \in \mathbb{R}^{n_{\mathrm{enc}}}$ is the parameter vector of the encoder[own]~~. The encoder estimates all $6+n_{\bar{x}}$ initial states $\hat{x}_\tau$, which includes the measured positions. We estimate the positions instead of using the measured y_τ to keep the positions consistent with the estimated velocities.

The encoder uses a linear map with a nonlinear correction [21, Eq. (8)]:

$$\psi(\theta_{\mathrm{enc}}, \chi_\tau) = \begin{bmatrix} W^b \\ W^a \end{bmatrix} \chi_\tau + \tilde{\psi}(\chi_\tau), \tag{20}$$

where $W^b \in \mathbb{R}^{6\times 6(n+1)}$ and $W^a \in \mathbb{R}^{n_{\bar{x}}\times 6(n+1)}$ are the linear maps for the six physical statesψ_b[own] and the $n_{\bar{x}}$ additional statesψ_a[own], and $\tilde{\psi}$ is a neural network. The encoder parameters θ_{enc} are W^b, W^a and the parameters of $\tilde{\psi}$. ~~The first six outputs ψ_b estimate the physical states and the remaining $n_{\bar{x}}$ outputs ψ_a estimate the additional states.~~[own] ~~We use θ^a_{enc} for the parameters that affect only ψ_a: W^a and the additional-state rows of the output layer of $\tilde{\psi}$.~~[moved to IV-E, own] [**TODO:** VERIFY bib: `hoekstra2026encoder` = arXiv:2602.13108v1 (13 Feb 2026), Hoekstra, Györök, Tóth, Schoukens; matches PDF p. 1. Check for a published (IFAC) version before submission.] [from IV-B, M-p7] SUBNET~~, the encoder-based identification method of Beintema et al. [15],~~[M-p7] assumes that the input is independent of the noise [15, Sec. 2.1] and [own]that the system that generates the data is incrementally exponentially output stable [15, Cond. 1]. The identification method we have adapted from [5] relies on ~~this SUBNET assumption~~ these SUBNET assumptions[own] for its convergence and consistency guarantees [5, Sec. 5.5]. The open-loop gantry system does not satisfy ~~this assumption~~ the second[own]: a mismatch in the initial velocity will create a permanent position offset. In closed-loop the feedback controller is able to correct this offset~~, which is why each window is simulated in the closed loop of Section IV-B[own]~~.[end]

~~With $x^c_\tau = 0$, the encoder only has to initialise the states of the model. Kessels instead needs the system's controller state at the start of each window, either measured directly or calculated by simulating the controller from the start of the record [?]Remark 5.4]kessels2025ai.~~[moved to IV-B, M-p7] [from IV-B, M-p8] ~~The controller uses the output error to correct the model input within the training window, as in the closed loop system.~~[own] Each window starts from a new encoder estimate and $x^c_\tau = 0$, so the window is not affected by model errors before τ. ~~The validation[own] of Section IV-C[M-mail] is simulated in closed-loop and only initialised at the start.~~[own end]

The baseline parameters and the neural network can both correct the same model errors, making the optimal parameter values not unique [5, Sec. 5.2]. [**TODO:** Missing: Section V makes the network orthogonal to the linearised baseline sensitivity at fixed tuples, so the split is unique only to first order around β_{lin}, not a unique minimiser of (18). Candidate: once Section V states that local result, write "so that the split is unique to first order" with a pointer to it (basis: `04_obc.tex` (32); math check 2026-10-13, F1).]

For training we minimise Vin (18)[M-p8] with Adam~~ on minibatches of windows~~ . Each update uses a minibatch of windows[own], and the gradient is calculated over the full window. [from below, own] The parameters are selected on a separate validation set $\mathcal{D}_{\mathrm{val}}$.[own] ~~The error is the mean RMS simulation error divided by the standard deviation of the servo error, $\mathrm{NRMS}_{e,j}$, over the complete records of $\mathcal{D}_{\mathrm{val}}$ and axes, simulated in closed loop with the initial state estimated once by the encoder and $x^c = 0$.~~ The records of $\mathcal{D}_{\mathrm{val}}$ are simulated in closed-loop and only initialised at the start. The encoder uses the first $n+1$ samples for this, and the validation error is calculated over the remaining samples.[own] During the optimisation with Adam the parameters with the lowest validation error are kept.[end] ~~We then polish with L-BFGS on a fixed set of windows. After Adam,~~ From these parameters,[own] we continue to minimise V with L-BFGS on a fixed set of windows. We call this step polish~~.~~ , and the parameter updates from this step are only kept if they lower the validation error.[own][own] ~~This is adapted from the Adam and L-BFGS-B sequence of Drenth et al.~~ Drenth et al. also use Adam followed by L-BFGS-B[own] [17, Sec. 4.1]. Because every β satisfies (9), unconstrained L-BFGS works without a bound. ~~The parameters are selected on a separate validation set $\mathcal{D}_{\mathrm{val}}$. During the optimisation with Adam the parameters with the lowest validation error are kept. The records of $\mathcal{D}_{\mathrm{val}}$ are simulated in closed-loop and only initialised at the start. The encoder uses the first $n+1$ samples for this, and the validation error is calculated over the remaining samples.~~[moved to above, own] ~~We only keep the polish if it lowers the validation error.~~[own]

D. Initialisation

- Only the positions are measured, so an encoder estimates all $6+n_{\bar{x}}$ states at each window start from the input-output window (Beintema Sec. 3.1); the window ends at $\tau-1$ (2026 Eq. (22e), D-250)
- The encoder is a linear map plus a network correction, as encoder paper Eq. (8), with no separate bias (the output bias of $\tilde{\psi}$ carries any constant)
- The physical rows start data-based: the baseline is self-scheduled by the state Y, hence nonlinear; a reconstruction from a linearisation is accurate only near its linearisation point (encoder paper Sec. 3.1), and the paper suggests the data-based route for nonlinear baselines (Secs. 3.2, 4.4)
- The physical rows are fitted to a baseline simulation as in 2026 Eq. (30); the simulation runs in closed loop because the open-loop baseline drifts on X and Y (this work, D-251)
- The fitted physical rows only set the start and are trained jointly with θ_{base} (2026 Eq. (22)); least-squares start (D-271)

A fully random initialisation can make the model unstable and lead to poor optimisation results. We therefore start training from a model that behaves as the baseline [5, Sec. 5.4]. [**TODO:** Word different, look at Quinten his comment.]
The physical rows are initialised from the baseline model. The baseline is nonlinear, because solving[own] ~~Because solving[own]~~ (4) for $\ddot{q}$ requires $M(Y)^{-1}$ and Y is a state (Section III-B), ~~the baseline is nonlinear[own]~~. ~~To create a model-based state reconstruction map for the nonlinear baseline, a linearisation point has to be chosen, making the map only accurate around that point~~ Hoekstra et al.'s model-based state reconstruction map for the nonlinear baseline needs a linearisation point, so the map is only accurate around that point[own] [21, Sec. 3.1]. Hoekstra et al. therefore suggest fitting the encoder to states simulated with the baseline [21, Sec. 3.2, Eq. (35)], as their augmentation method does [5, Sec. 5.4.2]. We apply this approach and simulate the baseline model once before training to obtain the physical state at every sample $\mp k$[M-p8]. The recorded actuator forces u_{act} and positions y from ~~the training records~~ the records of $\mathcal{D}_N$[own] with the known controller in (17) are used to simulate the physical states in closed-loop:

$$\tilde{x}^{\text{n},0}_{k+1} = f^{\text{n}}_{\text{base}}\big(\theta^0_{\text{base}}, \tilde{x}^{\text{n},0}_k, \hat{u}^{\text{n},0}_k\big), \qquad \hat{y}^{\text{n},0}_k = C^{\text{n}}\tilde{x}^{\text{n},0}_k, \quad (21)$$

where $\tilde{x}^{\text{n},0}_k$ is the simulated baseline state, θ^0_{base} are the initial parameters, and $\hat{u}^{\text{n},0}_k$ is the closed-loop input from (17). [from below, own] We use the closed-loop input from (21), instead of the recorded input as in [5, Sec. 5.3].[end] We use $\hat{y}^{\text{n},0}_k$ as the model output ~~and set the controller state x^c to zero at the start of each record (see Section IV-B)~~[own]. For our closed-loop simulation, each record starts from the measured positions$q = P^{-\top}y$[own] in generalised coordinates, ~~$q = P^{-\top}y$, with zero velocities, zero added states and $x^c = 0$~~[own]. The velocities, the added states and x^c start at zero.[own] [**TODO:** Check what Hoekstra et al. use as the initial state of the baseline simulation that generates the encoder-fitting data (2026 Sec. 5.4.2, encoder paper Sec. 3.2, and their public code), and whether our records start at rest, which the zero velocities assume.] The physical rows of the encoder are then fitted to the simulated states by least squares [21, Eq. (33)]:

$$\big(W^{b,0}, b^{b,0}\big) \in \arg\min_{W^b,\, b} \frac{1}{|\mathcal{D}|}\sum_{\tau\in\mathcal{D}}\big\|W^b\chi^0_\tau + b - \tilde{x}^{\text{n},0}_\tau\big\|_2^2, \quad (22)$$

where $b \in \mathbb{R}^6$ is ~~an intercept~~ a constant offset[own]. [from below, own] ~~χ^0_τ is formed as we do in (19) using $\hat{y}^{\text{n},0}$ and $\hat{u}^{\text{n},0}$. $\mathcal{D}$ is the set of window ends τ at the training stride of 10 samples.~~ χ^0_τ is built as χ_τ in (19), with $\hat{y}^{\text{n},0}$ and $\hat{u}^{\text{n},0}$ instead of the recorded y^{n} and u^{n}. $\mathcal{D}$ is the set of samples τ where we fit the encoder in (22).[own end] Hoekstra et al. use this linear fit for LTI baselines and fit a neural network for nonlinear baselines [21, Eqs. (33), (35)]. ~~We use the linear fit as a linear restriction of their data-based initialisation, and learn the nonlinear correction $\tilde{\psi}$ during training.~~ We only fit the linear map W^b. The nonlinear correction $\tilde{\psi}$ is learned during training.[own] ~~The affine fit is computed by a single numerical-rank-truncated SVD solve in centred and standardised regressors, without gradient-based pre-training.[own]~~ ~~The intercept $b^{b,0}$ is placed in the output bias of the physical rows of $\tilde{\psi}$, whose output weights start at zero, so that~~ The output bias of the physical rows of $\tilde{\psi}$ starts at the offset $b^{b,0}$, and their output weights start at zero. This gives[own] $\psi_b = W^{b,0}\chi_\tau + b^{b,0}$ at the start. ~~The hidden layers of $\tilde{\psi}$ are drawn from $\mathcal{U}(-1/\sqrt{m}, 1/\sqrt{m})$, where m is the input size of the layer.[own]~~ ~~We write $\theta^{b,0}_{\text{enc}}$ for this start of θ^b_{enc}, which consists of W^b, the hidden layers of $\tilde{\psi}$ and its physical output rows, and use it to start minimising (18).[own]~~ ~~χ^0_τ is formed as we do in (19) using $\hat{y}^{\text{n},0}$ and $\hat{u}^{\text{n},0}$. $\mathcal{D}$ is the set of window ends τ at the training stride of 10 samples.[moved to after (22), own]~~ ~~We use the closed-loop input from (21), instead of the recorded input as in [?]Eq. (30)]hoekstra2026lfr.[moved to after (21), own]~~ ~~The fitting loss in (22) and $\theta^{b,0}_{\text{enc}}$ are only used for initialisation. Subsequent training uses (18).[own]~~

~~We start training with the augmented model giving the same output as the baseline model from the same initial state, as posed by [?]Eq. (29)]hoekstra2026lfr.[M-p8]~~ [**Note:** I use the encoder for my augmented model, so i state: model from the same initial state. But i could leave it out.] ~~Therefore we~~ To start from the baseline, we[M-p8] zero the physical-state rows of the output layer of ϕ_{aug} and keep the added-state rows random:

$$\begin{aligned}
&\text{baseline:} && \beta = 0,\\
&\text{network:} && W^{\bar{x}}_L = 0, \quad b_L = 0, \quad W^{\bar{x}}_L \sim \mathcal{U}\big(-1/\sqrt{n_h}, 1/\sqrt{n_h}\big),\\
&\text{encoder:} && W^b = W^{b,0}, \quad \tilde{W}^{\bar{x}}_L = 0, \quad \tilde{b}^{\bar{x}}_L = b^{b,0},\\
& && W^a \sim \mathcal{U}(-a, a), \quad \tilde{W}^{\bar{x}}_L = 0, \quad \tilde{b}^{\bar{x}}_L = 0,\\
& && a = \sqrt{6/(n_{\bar{x}} + 6(n+1))},
\end{aligned} \quad (23)$$

[**Note:** (27) encoder row written out (was $\theta^{b,0}_{\text{enc}}$ from (26)); same values. Remove this note after checking.] where W_L and b_L are the output layer weights and bias of ϕ_{aug}, ~~with rows $W^{\tilde{x}}_L$ for $f^{\text{n}}_{\text{aug}}$ and $W^{\bar{x}}_L$ for $g^{\text{n}}_{\text{aug}}$~~. The rows $W^{\tilde{x}}_L$ and $W^{\bar{x}}_L$ are for $f^{\text{n}}_{\text{aug}}$ and $g^{\text{n}}_{\text{aug}}$, respectively[Dirk]. ~~$n_h$ is the number of neurons per hidden layer, and~~ n_h is the number of neurons in each hidden layer.[Dirk] ~~$\tilde{W}^{\bar{x}}_L$ and $\tilde{b}^{\bar{x}}_L$ are the added-state rows of the output layer weights and bias of $\tilde{\psi}$, respectively~~ For $\tilde{\psi}$, the output layer weights and bias are split into $\tilde{W}^{\tilde{x}}_L$, $\tilde{b}^{\tilde{x}}_L$ for the physical states and $\tilde{W}^{\bar{x}}_L$, $\tilde{b}^{\bar{x}}_L$ for the added states[Dirk]. Here $W \sim \mathcal{U}(-a, a)$ means ~~that every entry of W is drawn independently from the uniform distribution~~ each entry of W is drawn independently from a uniform distribution[Dirk] on $[-a, a]$. We initialise $W^{\bar{x}}_L$ and the hidden layers of $\phi_{\text{aug}}$and $\tilde{\psi}$[own] from $\mathcal{U}(-1/\sqrt{m}, 1/\sqrt{m})$, where m is the input size of the layer. We use Xavier initialisation for W^a, following [5, Eq. (31)]. We choose the uniform distribution with gain one, ~~giving $a = \sqrt{6/(n_{\bar{x}} + 6(n+1))}$~~[own]. At the start of training, $\theta_{\text{base}} = \theta^0_{\text{base}}$ and $f^{\text{n}}_{\text{aug}} = 0$. The augmented model and baseline model give the same $\tilde{x}$ and $\hat{y}$, given they start from the same physical state and receive the same input. The added states $\bar{x}$ are still updated by $g^{\text{n}}_{\text{aug}}$, as in [5, Eq. (29)].

E. Added-state initialisation

- Problem: the start (23) gives $\tilde{x}$ a model (f_{base}, W^b, output) and $\bar{x}$ none (random W^a, random added-state rows of ϕ_{aug}); V scores only the output and f_{aug} can lower the same error statically, so training can leave the omitted dynamics out of $\bar{x}$ (Venkatraman Sec. 2.1; Section VI tests it)
- PS is this work's, adapted from two-stage regression (Hefny Sec. 2) and its use before BPTT (Downey Sec. 4.2); it gives $\bar{x}$ a start from data, as W^b gives $\tilde{x}$ one from the baseline
- S1: a decoder on ψ_a predicts the current model's residual over τ to $\tau + H - 1$; the minimiser is the conditional mean, so $\bar{x}_{\tau|\tau}$ becomes a predictive state; trains θ_{p}, θ^a_{enc} only, one update after every update of V
- Stopping rule: ν on calibration windows ends S1 (plateau or cap) and decides whether S2 runs
- S2: once, one-step regression of $g^{\text{n}}_{\text{aug}}$ on frozen encoder pairs with the recorded input, which equals the loop input when the encoder reproduces y_{k_i}; overlapping windows (?)
- Hand-over: decoder discarded, (18) continues from ϑ_{PS}; deployed model, criterion and selection unchanged; $\bar{x}$ latent; $\mathcal{M}_{\text{PS}}$ named

[**Note:** pragmatisch bekijken, versimpelen. appendix probleem verplaatsen probeer makkelijker uit te leggen. misschien bepaalde elementen/dingen weg laten die niet echt nodig zijn voor het verhaal. pragmatisch mis niet dit hele stuk uit hoeven leggen.] from below, own The physical states ~~thus~~ own have a data-based start(22)own and a direct map to the output, while the added states have neither.end ~~The physical states of the baseline are initialised with the data-based fit (22) and updated through $f^{\text{n}}_{\text{base}}$, while the added-state rows W^a are randomly initialised.~~ The added states start from the random rows W^a and are updated through the random added-state rows of ϕ_{aug}.own ~~For the physical states their generalised coordinates q_k are directly mapped to actuator positions, $\hat{y}_k = P^\top q_k$ in (11).~~own ~~The added states are updated through the random added-state rows of ϕ_{aug}, without their initialisation or initial updates being derived from the baseline or the data.~~own ~~The output $\hat{y}_k = P^\top q_k$ contains no added state, and $\bar{x}_k$ enters~~ They enterown $\tilde{x}_{k+1}$ only through $f^{\text{n}}_{\text{aug}}$ in (18). ~~This means that for the added states V only updates their effect on the output through $f^{\text{n}}_{\text{aug}}$.~~M-p9 ~~From (18),~~ own V has no term that requires $\bar{x}$ to predict the ~~resulting~~ ownoutput error~~. This gives $f^{\text{n}}_{\text{aug}}$ the capability to lower~~ , so $f^{\text{n}}_{\text{aug}}$ can lowerown part of this error with $\tilde{x}$ and $\hat{u}$ alone. Minimising V can therefore reduce the error without capturing the missing dynamics in $\bar{x}$. [**TODO:** Section VII tests whether it does.] ~~The physical states thus have a data-based start and a direct map to the output, while the added states have neither.~~moved to start of IV-E, own

We therefore propose predictive-state (PS) initialisation, which is a data-driven initialisation for the added states. ~~Similar to W^b after its initialisation is done, the parameters are optimised through (18).~~own ~~PS is adapted from a two-stage regression of predictive-state learning [?]Sec. 2]hefny2015supervised, and it makes use of initialisation before backpropagation through time (BPTT) [?]Sec. 4.2]downey2017psrnn.~~moved to below, own PS has ~~three~~ twoown stages, each with its own ~~optimisation problem~~ cost functionown. ~~In S1,~~ In the first stage (S1),own ~~(18) is minimised as before, and after each of its updates the added-state outputs of the encoder are trained to predict the residual of the model~~ the encoder's added-state outputs are trained to predict the residual of the augmented modelown. ~~In S2,~~ In the second stage (S2),own $g^{\text{n}}_{\text{aug}}$ is trained to predict the next added state. After S2, ~~(18) is minimised from the resulting parameters~~ we continue minimising Vown. from above, own PS is adapted from a two-stage regression of predictive-state learning [22, Sec. 2]~~, and it makes use of initialisation before backpropagation through time (BPTT)~~ , which Downey et al. use to initialise a recurrent network before backpropagation through time (BPTT)own [23, Sec. 4.2].end ~~The first stage (S1)~~ S1own predicts the residual of the model, instead of features of future observations as in [22]. ~~The second stage (S2)~~ S2own trains the nonlinear state update instead of a ~~linear operator~~ linear mapown. We continue minimising V during the first stage~~, while Downey et al. finish their initialisation before starting BPTT [?]Sec. 4.2]downey2017psrnn~~own. Fig. 5 shows which samples are used for ~~both stages~~ each stageDirk.

~~Stage S1 trains the encoder's added-state outputs to predict the residual of the augmented model (11) using~~ In S1, the residual is computed withown the model parameters ϑ from the latest ~~V update step~~ update of Vown. from IV-C, own ~~We use θ^a_{enc} for the parameters that affect only ψ_a: W^a and the additional-state rows of the output layer of $\tilde{\psi}$.~~ The encoder parameters for the added states are W^a and the added-state rows $\tilde{W}^{\bar{x}}_L$, $\tilde{b}^{\bar{x}}_L$ of the output layer of $\tilde{\psi}$. We call these θ^a_{enc}.own end ~~Each update of V is followed by one S1 update with its own Adam optimiser.~~own For the S1 update we select a minibatch $\mathcal{B}$ of samples τ from ~~the training data~~ $\mathcal{D}_N$own. A subset of records is left out of S1 and S2 to evaluate the performance of each stage.Dirk ~~The training data~~

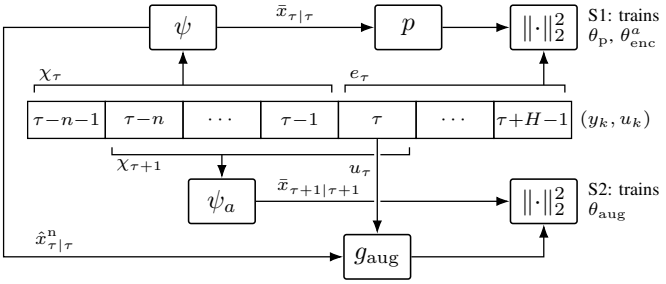


Fig. 5: Samples used by PS on one record, with the layout adapted from [22, Fig. 2]. In S1 (top), the encoder ψ estimates $\hat{x}^{\text{n}}_{\tau|\tau}$ from the window χ_τ. The decoder p then predicts the residual e_τ of the next H samples from the added states $\bar{x}_{\tau|\tau}$ (24). In S2 (bottom), g_{aug} predicts the next added state $\bar{x}_{\tau+1|\tau+1}$ from $\hat{x}^{\text{n}}_{\tau|\tau}$ and the recorded u_τ (26). The same encoder gives $\bar{x}_{\tau+1|\tau+1}$ from the window $\chi_{\tau+1}$, shifted by one sample.

[**Note:** d is also a baseline parameter, need to change]

~~is divided into excitation classes, and for classes with at least three records one is left out of S1 and S2 to evaluate ν (??). The left out data is still used in V.~~[moved to the u paragraph, M-mail] ~~For each sample τ, the encoder uses the last $n+1$ samples up to $\tau-1$, χ_τ, to estimate the initial state $\hat{x}^{\mathrm{n}}_{\tau|\tau}$.~~[own] From ~~this state~~ the encoder state $\hat{x}^{\mathrm{n}}_{\tau|\tau}$[own] the augmented model is simulated in the closed loop of Section IV-B. We calculate the residual$y-\hat{y}$[own] ~~from~~ over[own] the next $H=n+1$ samples, from τ to $\tau+H-1$. The top of Fig. 5 shows the window χ_τ and the samples of the residual e_τ. ~~We have chosen to use the same number of samples as the encoder window.~~[own] Then a temporary decoder ~~d~~ p[own], a neural network with one hidden layer,[own] with only the added states as inputs $\bar{x}_{\tau|\tau}=\psi_a(\theta_{\mathrm{enc}},\chi_\tau)$ is used to predict this residual:

$$\min_{\theta_{\mathrm{p}},\,\theta^a_{\mathrm{enc}}} \frac{1}{|\mathcal{B}|\,H\,n_y}\sum_{\tau\in\mathcal{B}}\left\|p\big(\theta_{\mathrm{p}},\psi_a(\theta_{\mathrm{enc}},\chi_\tau)\big)-e_\tau\right\|_2^2,$$
$$e_\tau=\frac{1}{\sigma_e}\left[\,(y^{\mathrm{n}}_\tau-\hat{y}^{\mathrm{n}}_{\tau|\tau})^\top\ \cdots\ (y^{\mathrm{n}}_{\tau+H-1}-\hat{y}^{\mathrm{n}}_{\tau+H-1|\tau})^\top\right]^\top, \tag{24}$$

[**Note:** (28) reordered (minimisation first, as (22) and (30)); decoder form moved to the appendix. Remove this note after checking.] where $\hat{y}^{\mathrm{n}}_{k|\tau}$ is the output from the closed-loop simulation of Section IV-B, starting at $\hat{x}^{\mathrm{n}}_{\tau|\tau}$. This simulated output is kept fixed during the S1 update. ~~$\mathcal{B}$ is the minibatch of samples τ and~~[own] σ_e is the RMS of all residual values in $\mathcal{B}$. ~~The decoder $d:\mathbb{R}^{n_{\bar{x}}}\to\mathbb{R}^{Hn_y}$ has parameters $\theta_{\mathrm{p}}=(W_{\mathrm{p},1},b_{\mathrm{p},1},W_{\mathrm{p},2},b_{\mathrm{p},2})$, with $W_{\mathrm{p},1}\in\mathbb{R}^{n_h\times n_{\bar{x}}}$ and $W_{\mathrm{p},2}\in\mathbb{R}^{Hn_y\times n_h}$. Both $W_{\mathrm{p},2}$ and $b_{\mathrm{p},2}$ are initially set to zero, so that the decoder starts by predicting zero residual.~~[moved to App. ??, own] Only θ_{p} and θ^a_{enc} are updated by S1. The parameters of ϑ, including θ^a_{enc} are still trained through V.
The expected squared error is minimised by the conditional mean $\mathbb{E}[e_\tau \mid \chi_\tau]$. The decoder only sees up to $\bar{x}_{\tau|\tau}$, not the full history χ_τ, this ~~forces~~ encourages[own] the encoder to use the added states to predict the future residual. [**TODO:** Instead of using features of future observations as Hefny et al. [22, Sec. 2] for predictive-state learning, we use the model residual.] ~~To avoid correlation between the history and future noise Hefny et al. separate the past and future samples [?]Sec. 2]hefny2015supervised. The encoder window ends at $\tau-1$, while the residual starts at τ, so no measurement sample is reused on both sides. With filtered measurement noise, this separation does not guarantee noise independence.~~[moved to App. ??, own] [**TODO:** Venkatraman et al. use a similar approach by predicting future observations from the internal states during training [24, Sec. 3].] ~~For the PS implementation the decoder predicts the model residual and is removed after S1.~~[own]

~~The training data is divided into excitation classes, and for classes with at least three records one is left out of S1 and S2 to evaluate S1. The left out data is still used in V. To evaluate how well $\bar{x}_{\tau|\tau}$ performs at predicting the residual, a fixed set of windows $\mathcal{C}$ from the held out data is used. For this set the squared prediction error of the decoder divided by the squared residual is computed, [equation (??) moved to Appendix ??]~~

~~S1 is stopped when a maximum number of updates is reached or when ν stops improving, or at an early check when ν is above $\nu_{\max}$. S2 starts only if ν is below a threshold $\nu_{\max}$, else the decoder is removed and training continues. In both cases the encoder changes from S1 are kept. The threshold rule for S2 is given in Appendix ??.~~[moved to App. ??, own] ~~We use the left out records to decide when S1 ends and whether S2 runs (Appendix ??).~~ S1 ends when the decoder's prediction error on the left out records stops improving, and S2 runs only if the decoder explains at least 10 % of the residual there.[own] The left out records are still used to minimise V.[own] ~~Written as one procedure, S1 alternates the two problems: for~~ During S1, each Adam step on V is followed by one Adam step on (24). For[own] $i=0,1,\ldots,i_1-1$, this gives[own]

$$\begin{aligned}\vartheta^{(i+1)}&=\mathcal{A}_V\big(\vartheta^{(i)}\big),\\ \big(\theta_{\mathrm{p}}^{(i+1)},\theta_{\mathrm{enc}}^{a,(i+1)}\big)&=\mathcal{A}_{\mathrm{S1}}\big(\theta_{\mathrm{p}}^{(i)},\theta^a_{\mathrm{enc}}\text{ of }\vartheta^{(i+1)}\big),\end{aligned} \tag{25}$$

where ~~$\mathcal{A}_V$ is one Adam step on (18), $\mathcal{A}_{\mathrm{S1}}$ one Adam step on (24) with its own Adam optimiser, the updated θ^a_{enc} is written back into $\vartheta^{(i+1)}$, and i_1 is the last update of S1~~ $\mathcal{A}_V$ and $\mathcal{A}_{\mathrm{S1}}$ are one Adam step on (18) and (24), each with its own Adam optimiser. Both steps update θ^a_{enc}, so the θ^a_{enc} from $\mathcal{A}_{\mathrm{S1}}$ replaces the one in $\vartheta^{(i+1)}$, and i_1 is the last update of S1[own]. [**TODO:** Missing: basis of the PS switch and stop rules (S1 plateau and cap, screen update and threshold, S2 plateau and cap, snapshot sizes, calibration split as the last record of each class with three or more records). Candidate: rows of the hyperparameter table in Section VI, all HEURISTIC (`PS2Spec`; `PS2-METHOD.md` Sec. 5), noting that the S1 cap, not the plateau, ended S1 in every run so far.]

S1 has only trained the part of the encoder that initialises the added states $\bar{x}_{\tau|\tau}$, ~~$g^{\mathrm{n}}_{\mathrm{aug}}$ is only updated by V~~[own]. S2 trains the state update$g^{\mathrm{n}}_{\mathrm{aug}}$[own] itself~~. In S2 $g^{\mathrm{n}}_{\mathrm{aug}}$ is trained~~ ,[own] to predict the next added state $\bar{x}_{k+1}$ from the full state $\hat{x}^{\mathrm{n}}_k$ and the input u^{n}_k. For S2 we select [own]samples k_i, ~~$i=1,\ldots,N_{\mathcal{P}}$,~~[own] from ~~the training data $\mathcal{D}_N$~~[own] ~~are selected,~~[own] ~~except the same held-out records from S1 are left out~~ without the left out records[own]. For each sample the encoder estimates $\hat{x}^{\mathrm{n}}_{k_i|k_i}$ from χ_{k_i} and $\bar{x}_{k_i+1|k_i+1}$ from the next window χ_{k_i+1}. ~~Fig. 5 shows an example with $k_i=\tau$~~ In Fig. 5, k_i is the sample τ[own]. ~~The window χ_τ at the top is used for $\hat{x}^{\mathrm{n}}_{k_i|k_i}$, and the shifted window at the bottom is used for $\bar{x}_{k_i+1|k_i+1}$.~~[own] We calculate ~~these pairs~~ $\hat{x}^{\mathrm{n}}_{k_i|k_i}$ and $\bar{x}_{k_i+1|k_i+1}$[own] once after S1~~, this set $\mathcal{P}$ in (26) is kept fixed~~ . Together with the recorded inputs $u^{\mathrm{n}}_{k_i}$, they form the fixed set $\mathcal{P}$ in (26)[own].
During S2 the model is not simulated in closed-loop and V is ~~paused~~ not minimised[own]. We instead train $g^{\mathrm{n}}_{\mathrm{aug}}$ on $\mathcal{P}$ using one-step regression with the recorded actuator force $u^{\mathrm{n}}_{k_i}$ as input, and a separate Adam optimiser~~:~~ .[own] ~~from below, own~~ At the start of a window, the recorded force equals the closed-loop input of V when the encoder reproduces y_{k_i} ~~(Appendix ??)~~ , by (15)

with $x_\tau^c = 0$[own]. [end] S2 solves[own]

$$\mathcal{P} = \left\{(\hat{x}^{\mathrm{n}}_{k_i|k_i},\, u^{\mathrm{n}}_{k_i},\, \bar{x}_{k_i+1|k_i+1})\right\}_{i=1}^{N_{\mathcal{P}}},$$
$$\hat{x}^{\mathrm{n}}_{k_i|k_i} = \psi(\theta_{\mathrm{enc}}, \chi_{k_i}), \quad \bar{x}_{k_i+1|k_i+1} = \psi_a(\theta_{\mathrm{enc}}, \chi_{k_i+1}),$$
$$\min_{\theta_{\mathrm{aug}}} \frac{\sum_{i\in\mathcal{B}_{\mathcal{P}}} \left\| g^{\mathrm{n}}_{\mathrm{aug}}(\theta_{\mathrm{aug}}, \hat{x}^{\mathrm{n}}_{k_i|k_i}, u^{\mathrm{n}}_{k_i}) - \bar{x}_{k_i+1|k_i+1} \right\|_2^2}{\sum_{i\in\mathcal{B}_{\mathcal{P}}} \sum_{j=1}^{n_{\bar{x}}} \left(\bar{x}_{j,k_i+1|k_i+1} - m_j\right)^2}, \tag{26}$$

where θ_{enc} is not updated during S2, $\mathcal{B}_{\mathcal{P}} \subset \mathcal{P}$ is a minibatch, ~~$\bar{x}_{j,\cdot}$~~ $\bar{x}_{j,k_i+1|k_i+1}$[own] is the jth added state and m_j is the mean of $\bar{x}_{j,k_i+1|k_i+1}$ over $\mathcal{B}_{\mathcal{P}}$. ~~The cost is the fraction of the variance in the next added states that is not explained by the prediction.~~ We divide the prediction error by the variance of $\bar{x}_{k_i+1|k_i+1}$ over $\mathcal{B}_{\mathcal{P}}$ to keep the cost independent of the scale of $\bar{x}$.[own] ~~At the start of a window, the recorded force equals the closed-loop input of V when the encoder reproduces y_{k_i} (Appendix ??).~~[moved to above (30), own]

~~The parameters with the lowest cost on the separate set of pairs from the held-out records are kept.~~ We keep the network parameters θ_{aug} with the lowest cost (26) on the left out records.[own] ~~For S2 we use the recorded actuator force $u^{\mathrm{n}}_{k_i}$ applied to the system at k_i. The encoder estimates $\bar{x}_{k_i+1|k_i+1}$ from χ_{k_i+1}, using the data up to k_i+1. At the start of a window in (15) $x^c = 0$, resulting in the model input [equation moved to Appendix ??] when the encoder reproduces y_{k_i} this equals the recorded input, because at this sample only the controller feedthrough D_{fb} acts on the output error. For S2 $g^{\mathrm{n}}_{\mathrm{aug}}$ is therefore trained with approximately the input it receives at that sample in the closed-loop simulation of V.~~[moved to App. ??, own] S2 updates all parameters θ_{aug} of ϕ_{aug}. ~~The cost only depends on the added-state rows, so the physical rows of W_L and b_L are not changed. The hidden layers of ϕ_{aug} are shared by $f^{\mathrm{n}}_{\mathrm{aug}}$ and $g^{\mathrm{n}}_{\mathrm{aug}}$, so S2 also updates $f^{\mathrm{n}}_{\mathrm{aug}}$.~~ The physical rows of W_L and b_L stay unchanged, because (26) only depends on the added-state rows, but the shared hidden layers also change $f^{\mathrm{n}}_{\mathrm{aug}}$.[own] [**TODO:** The encoder windows at k_i and k_i+1 share measurement samples, so noise enters both sides of the S2 regression. Hefny et al. discuss how correlated noise can bias such a fit [22, Sec. 2]. We do not correct for this effect in S2.] ~~When S2 is completed the optimisation continues minimising (18), starting from~~ After S2, we continue minimising (18) from[own]

$$\vartheta_{\mathrm{PS}} = \left(\beta^{(i_1)},\, \theta^{\mathrm{S2}}_{\mathrm{aug}},\, \theta^{(i_1)}_{\mathrm{enc}}\right), \tag{27}$$

followed by the ~~L-BFGS~~[own] polish and the [own] selection of Section IV-C. Here $\beta^{(i_1)}$ and $\theta^{(i_1)}_{\mathrm{enc}}$ are the parameters after S1 and $\theta^{\mathrm{S2}}_{\mathrm{aug}}$ ~~contains the parameters~~[own] after S2. When S2 is skipped, ϑ_{PS} contains the parameters ~~at the end of~~ after[own] S1. PS ~~has only changed~~ only changes[own] the starting point of minimising V ~~with its initialisation phases~~[own], not what is ~~being~~ [own] minimised. ~~The models with and without PS have the same number of parameters, since the decoder is removed.~~ PS adds no parameters to the augmented model, because the decoder is removed after S1.[own]

V. Interpretability by Orthogonal Construction

- Goal from Section III: an estimate of θ_{base} that is unique under stated conditions; names IV-A and IV-B (style profile)
- Contribution: the extension Györök 2026 Sec. 6 leaves open: a state-space baseline without full-state measurement, here nonlinear in its parameters and identified in closed loop (this work)

[Changes: black = original, blue = original, moved here (from … end), green = changed or new, ~~struck~~ = deleted ("moved to" = moved, not deleted), superscript = comment id.]

In this section, we make the estimate of θ_{base} unique. The parameter pair $(\theta_{\mathrm{base}}, \theta_{\mathrm{aug}})$ minimising the loss function of Section IV-C can be non-unique: different pairs can give the same fit [8, Sec. 3.1]. ~~Meaning that the rigid-body dynamics that the baseline parameters θ_{base} are able to explain from the data, can instead be represented by a neural network correction.~~ Then θ_{base} is not interpretable as defined in Section I.[Jan] To prevent the neural network from correcting what θ_{base} can represent we introduce orthogonality by construction [8]. Orthogonality here is defined as the neural network correction to the physical states having no component in the directions in which a change of θ_{base} is able to change the next state predicted by the baseline. This gives us a unique estimate of θ_{base} up to first order~~, but it does not guarantee recovering the true baseline parameters.~~ . This estimate equals the true parameters only if the missing dynamics themselves are orthogonal to these directions.[M-p15]

Györök et al. have constructed this for input-output baseline models that are linear in their parameters [8, Sec. 2]. We extend this to state-space models nonlinear in their parameters, specifically for our gantry system.

A. Orthogonal-by-construction parametrisation

- Györök's construction as adopted: baseline linear in its parameters, regressor at measured input-output data (past outputs, past and current inputs), stacked over the training data; lower case per sample, upper case stacked (2026 Eqs. (1), (2), (5))
- θ_{aux} is the least-squares fit of the network output by the regressor; its fitted part is removed and the remainder is orthogonal to Φ on the training data (2026 Eqs. (8) to (11), Lemma 3)
- θ_{aux} exists for full column rank of Φ: identifiable baseline, weakly persistently exciting input (2026 Eqs. (6), (7), Assumption 1)
- Data-driven: orthogonality holds on the data used, the projection depends on their distribution; any fixed set may be used, the analysis uses the whole training set (2026 Sec. 3.2, text after Eqs. (12), (13))
- Uniqueness of $\hat{\theta}_{\mathrm{b}}$ shown under Assumption 1 and exact reproduction of the noise-free dynamics on the same training data (2026 Sec. 4.2, Assumption 6)

For a baseline linear in its parameters, $\hat{y}_k = \varphi(x_k)\theta_{\mathrm{b}}$, changing θ_{b} changes the model output $\hat{y}_k$ by a linear combination of

the columns of the regressor $\varphi(x_k)$ [8, Eqs. (2), (39)]. Györök et al. subtract from the neural network output its least-squares fit by the stacked regressor [8, Eqs. (8), (9), (11), Lemma 3]:

$$\begin{aligned} \hat{Y} &= \Phi\theta_{\mathrm{b}} + \tilde{F}^{\mathrm{ANN}}, \\ \tilde{F}^{\mathrm{ANN}} &= F^{\mathrm{ANN}} - \Phi\theta_{\mathrm{aux}}, \\ \theta_{\mathrm{aux}} &= \left(\Phi^{\top}\Phi\right)^{-1}\Phi^{\top}F^{\mathrm{ANN}}, \\ 0 &= \Phi^{\top}\tilde{F}^{\mathrm{ANN}}, \end{aligned} \tag{28}$$

where x_k is the vector of past measured outputs and past and current inputs [8, Eq. (1)], $\hat{Y}$ stacks the predicted outputs over ~~the training data~~ $\mathcal{D}_N$[own], Φ is the stacked regressor, F^{ANN} stacks the parallel network outputs [8, Eq. (5)] and $\tilde{F}^{\mathrm{ANN}}$ is the projected network output [8, Eqs. (9), (10)].
The projected network output $\tilde{F}^{\mathrm{ANN}}$ is orthogonal to the regressor [8, Lemma 3]. Calculating θ_{aux} requires $(\Phi^{\top}\Phi)^{-1}$, which exists only if Φ has full column rank [8, Assumption 1]. Therefore different values of θ_{b} are unable to give the same stacked prediction on ~~the training data~~ $\mathcal{D}_N$[own]. ~~Györök et al. call an input with this property~~ An input with this property is[Dirk] weakly persistently exciting [8, Eq. (6), Sec. 4.3]. This also requires an identifiable baseline, where different values of θ_{b} give different functions $\varphi(\cdot)\theta_{\mathrm{b}}$.

B. Extension to the gantry

- Three differences, each changing one element of (28): regressor, data, prediction (this work)
- Regressor: Jacobian of the normalised transition in β at β_{lin}, since the baseline is nonlinear through $M(Y)^{-1}$; β because only the ten combinations are identifiable (adapted from Györök 2025 Sec. 4, Eq. (15); 2026 Assumption 1; Section II-C; D-190, D-229); offset column Γ included (D-257)
- Data: reference tuples reconstructed from the measured positions, regressor at the data as in 2026 Eq. (5) rather than at a simulated baseline (2025 Sec. 3); each with a successor; built once before training (D-252, D-254)
- Projection on the stacked tuples, with the orthogonality of the section opening as its last line; holds for the stack at $\bar{x} = 0$ (adapted from 2026 Eqs. (8) to (11), Lemma 3)
- Prediction: the correction at every step of the closed-loop state update, physical rows only; coefficient differentiated (2026 Eqs. (9), (10), (13); added states this work)

Our baseline violates three requirements of the projection in (28). ~~Its~~ First, its state update is nonlinear in the parameters, because the next state update depends on $M(Y)^{-1}$ in (7), violating the linear requirement for the regressor. Instead of the regressor the Jacobian of the baseline state update is taken with respect to β, as in the orthogonal regularisation method of Györök et al. [25, Sec. 4, Eq. (15)]. Like the regressor, the stacked Jacobian must have full column rank. This requires identifiable parameters, which is why we use β(Section III-C)[JsV1]. The Jacobian is taken at a linearisation point β_{lin} as

$$\varphi(\beta_{\mathrm{lin}}, \tilde{x}^{\mathrm{n}}, u^{\mathrm{n}}) = \left.\frac{\partial f^{\mathrm{n}}_{\mathrm{base}}\left(\theta_{\mathrm{base}}(\beta), \tilde{x}^{\mathrm{n}}, u^{\mathrm{n}}\right)}{\partial\beta}\right|_{\beta=\beta_{\mathrm{lin}}}, \tag{29}$$

where $\beta_{\mathrm{lin}} \in \mathbb{R}^{10}$ and $\varphi : \mathbb{R}^{10} \times \mathbb{R}^{6} \times \mathbb{R}^{3} \to \mathbb{R}^{6\times 10}$. ~~The linearisation also leaves an offset that does not depend on β~~ A first-order expansion of the baseline state update gives[Dirk] [25, Eq. (16)]:

$$\begin{aligned} f^{\mathrm{n}}_{\mathrm{base}}\left(\theta_{\mathrm{base}}(\beta), \tilde{x}^{\mathrm{n}}, u^{\mathrm{n}}\right) &\approx \varphi\,\beta + \gamma, \\ \gamma &= f^{\mathrm{n}}_{\mathrm{base}}\left(\theta_{\mathrm{base}}(\beta_{\mathrm{lin}}), \tilde{x}^{\mathrm{n}}, u^{\mathrm{n}}\right) - \varphi\,\beta_{\mathrm{lin}}, \end{aligned} \tag{30}$$

where the first term is linear in β and[Dirk] $\gamma : \mathbb{R}^{10} \times \mathbb{R}^{6} \times \mathbb{R}^{3} \to \mathbb{R}^{6}$, ~~so that $\varphi\beta_{\mathrm{lin}} + \gamma$ is the next state predicted by the baseline at β_{lin}~~ is an offset independent of β[Dirk]. Györök et al. add this offset as an extra column to the Jacobian, to ~~include all components of the baseline model in the orthogonality~~ promote orthogonality between the neural network and the next states predicted by the baseline[Dirk] [25, Eq. (18)]. We include it as well, ~~so the neural network has no component along the stacked next states predicted by the baseline~~[Dirk]. ~~The offset does not change the first-order estimate (35).~~[moved to V-C, M-mail] ~~The offset does not fully prevent negation [?]Sec. 5.3.1.3]kessels2025ai, where the neural network cancels the baseline response.~~[own] **[Note:** Removed: the sentence on negation. Negation is not measured or reported anywhere in the thesis, and in the final runs the unconstrained network hardly negates the baseline either.]

~~The~~ Second, the Jacobian also depends on the state, while the regressor of Györök et al. uses only measured inputs and outputs [8, Eqs. (1), (2)]. Since only the positions are measured, we evaluate the Jacobian at these positions with the velocities reconstructed from them:

$$\begin{aligned} \tilde{x}_i &= \mathrm{col}(q_i, \dot{q}_i), \qquad q_i = P^{-\top} y_i, \\ \dot{q}_i &= \frac{-q_{i+2} + 8q_{i+1} - 8q_{i-1} + q_{i-2}}{12\,T_s}, \end{aligned} \tag{31}$$

where y_i is the measured actuator position at sample i. Each tuple $(\tilde{x}_i, \bar{x}_i, u_i)$ uses the recorded actuator force u_i and $\bar{x}_i = 0$, with the physical state and input normalised to $\tilde{x}^{\mathrm{n}}_i$ and u^{n}_i using (12). The tuples are calculated once before training, using all samples for which (31) can be calculated at i and at the next sample $i+1$.
Stacking the Jacobian, the offset and the network output over the tuples, the projection of (28) becomes

$$\begin{aligned} \Phi_{\Gamma} &= \mathrm{col}\left([\varphi \;\; \gamma](\beta_{\mathrm{lin}}, \tilde{x}^{\mathrm{n}}_i, u^{\mathrm{n}}_i)\right)_{i\in\mathcal{I}}, \\ F^{\mathrm{n}}_{\mathrm{aug}}(\theta_{\mathrm{aug}}) &= \mathrm{col}\left(f^{\mathrm{n}}_{\mathrm{aug}}(\theta_{\mathrm{aug}}, \mathrm{col}(\tilde{x}^{\mathrm{n}}_i, 0), u^{\mathrm{n}}_i)\right)_{i\in\mathcal{I}}, \\ \theta_{\mathrm{aux}}(\theta_{\mathrm{aug}}) &= \left(\Phi_{\Gamma}^{\top}\Phi_{\Gamma}\right)^{-1}\Phi_{\Gamma}^{\top}F^{\mathrm{n}}_{\mathrm{aug}}(\theta_{\mathrm{aug}}), \\ \tilde{F}^{\mathrm{n}}_{\mathrm{aug}}(\theta_{\mathrm{aug}}) &= F^{\mathrm{n}}_{\mathrm{aug}}(\theta_{\mathrm{aug}}) - \Phi_{\Gamma}\,\theta_{\mathrm{aux}}(\theta_{\mathrm{aug}}), \\ 0 &= \Phi_{\Gamma}^{\top}\tilde{F}^{\mathrm{n}}_{\mathrm{aug}}(\theta_{\mathrm{aug}}), \end{aligned} \tag{32}$$

where $\mathcal{I}$ is the set of tuples, $N = |\mathcal{I}|$, $\Phi_{\Gamma} = [\Phi\ \Gamma] \in \mathbb{R}^{6N\times 11}$ stacks φ and γ evaluated at β_{lin}, $F^{\mathrm{n}}_{\mathrm{aug}}, \tilde{F}^{\mathrm{n}}_{\mathrm{aug}} \in \mathbb{R}^{6N}$ and $\theta_{\mathrm{aux}} \in \mathbb{R}^{11}$. The projected network output $\tilde{F}^{\mathrm{n}}_{\mathrm{aug}}$ therefore

has no component along any column of Φ_Γ, which satisfies our orthogonality definition at the start of this section. This holds for the stacked tuples at $\bar{x} = 0$, not for each tuple separately. ~~We~~ Third, we use a state-space model predicting the next state instead of the output. The fitted part is removed from the neural network correction of the physical state at every step in the closed-loop simulation, adapting the output correction in [8, Eqs. (9), (13)][JsV10]. At each step, we subtract the Jacobian and offset multiplied by θ_{aux} from the network correction, evaluated at the current state and input. ~~Györök et al. apply the same correction to the predicted output [?]Eqs. (9), (13)]gyorok2026obc.~~[JsV10] The baseline has no added states, so its Jacobian and offset are zero in the added-state rows and $g^{\text{n}}_{\text{aug}}$ is orthogonal to it by definition. We therefore only correct the physical rows:

$$\begin{aligned} f^{\text{n}}_{\text{obc}}(\theta_{\text{aug}}, \tilde{x}^{\text{n}}, u^{\text{n}}) &= [\varphi \quad \gamma](\beta_{\text{lin}}, \tilde{x}^{\text{n}}, u^{\text{n}})\, \theta_{\text{aux}}(\theta_{\text{aug}}), \\ \tilde{x}^{\text{n}}_{k+1|\tau} &= f^{\text{n}}_{\text{base}} + f^{\text{n}}_{\text{aug}} - f^{\text{n}}_{\text{obc}}, \\ \bar{x}_{k+1|\tau} &= g^{\text{n}}_{\text{aug}}, \end{aligned} \tag{33}$$

where $f^{\text{n}}_{\text{obc}} : \mathbb{R}^{n_{\text{aug}}} \times \mathbb{R}^6 \times \mathbb{R}^3 \to \mathbb{R}^6$ is evaluated at $(\tilde{x}^{\text{n}}_{k|\tau}, \hat{u}^{\text{n}}_{k|\tau})$, and $f^{\text{n}}_{\text{base}}$, $f^{\text{n}}_{\text{aug}}$ and $g^{\text{n}}_{\text{aug}}$ ~~take the arguments of~~ are evaluated as in[Dirk] (18). We differentiate θ_{aux} with respect to θ_{aug}, using its expression in [8, Eq. (8)], ~~so the network is trained~~ to train the network[Dirk] using the gradient of its projected output.

C. Identification and recovery

- $\mathcal{M}_{\text{OBC}}$: the augmented model of Section III with the PS start and the correction; identified by (18) with (33) as the physical-state update, everything of Section III-C reused (this work)
- Rules, each from a dependence: θ_{aux} re-solved at every evaluation; β_{lin} per epoch, a fixed objective per epoch while the point follows the estimate; 2025 Sec. 4 options not taken (cost, detuned start), Remark 2 adapted (D-190, D-254); rebuild after restore, polish compared with the rebuilt model (D-253, revnote)
- Recovery: Ξ (34), the unmodelled terms of 2026 Eqs. (14), (15) plus reconstruction error and noise; the one-step error and its two first-order minimisers (35); form of 2026 Eq. (21) without the exact fit of Assumption 6; true iff $\Phi^\top \Xi = 0$ (Condition 4, Remark 5); Γ does not enter (D-257)
- Limits: a one-step surrogate at $\bar{x} = 0$, not the closed-loop optimum; negation is treated in IV-B
- Measures: cos, ρ, Π (36) (supervisor: orthogonality is measured by the inner product); negation by the cosine with the baseline's one-step increment (D-257); c_j dropped (correlated columns of Φ; the per-parameter bias is (35))

The augmented model is identified as in Section IV-C, with the added-state initialisation of Section IV-E. With orthogonality by construction the state update in (18) is instead the projected state update (33). Since θ_{aux} depends on θ_{aug}, we solve (32) at every evaluation of V. At the start of each epoch β_{lin} is set to the current β, and ~~it~~[Dirk] is held fixed during the epoch. ~~Györök et al. update β_{lin} at every iteration, which requires rebuilding Φ_Γ at each evaluation of V [?]Sec. 4]gyorok2025l4dc. The other proposed method is to keep β_{lin} fixed at the nominal parameters, assuming these are close to the true parameters [?]Sec. 4]gyorok2025l4dc.~~ Györök et al. propose updating β_{lin} at every iteration or keeping it fixed at the nominal parameters [25, Sec. 4].[M-mail] ~~Training as stated in Section VI starts~~ To allow training to start[M-mail] from detuned parameters without rebuilding Φ_Γ at each evaluation of V[Dirk], we instead adapt their recomputation at the start of each epoch [25, Remark 2]. ~~We load the best checkpoint and rebuild Φ_Γ using its β. The polish minimises the same projected objective, with Φ_Γ held fixed at this β. The polish is only kept if its validation error is lower than the selected checkpoint's.~~[M-mail] [**Note:** Final OBC results pending (polish reruns of R1 and R2, 2026-10-20, final evaluation not yet run). R1 and R13: the polish is kept, validation NRMS_e from 7.751476×10^{-2} to 7.746424×10^{-2} and from 2.926736×10^{-1} to 2.925875×10^{-1}. R2: the polish did not change the model, validation NRMS_e 7.834410×10^{-2} before and after (checkpoint 7.834413×10^{-2}).] The projection does not guarantee that the true parameters $\beta^\star \in \mathbb{R}^{10}$ are recovered. That depends on whether ~~what the baseline misses~~ the missing dynamics[Dirk] on the tuples ~~is itself~~ are[Dirk] orthogonal to the stacked Jacobian:

$$\Xi = \operatorname{col}\big(\tilde{x}^{\text{n}}_{i+1} - f^{\text{n}}_{\text{base}}(\theta_{\text{base}}(\beta^\star), \tilde{x}^{\text{n}}_i, u^{\text{n}}_i)\big)_{i \in \mathcal{I}}, \tag{34}$$

where $\Xi \in \mathbb{R}^{6N}$ contains the unmodelled terms of [8, Eqs. (14), (15)], the reconstruction error of (31) and the measurement noise. To first order around $\beta^\star$, the one-step error on the tuples is $\Xi - \Phi(\beta - \beta^\star) - \tilde{F}^{\text{n}}_{\text{aug}}$. Minimising the squared one-step error over β gives $\hat{\beta}$. Without the projection, $F^{\text{n}}_{\text{aug}}$ replaces $\tilde{F}^{\text{n}}_{\text{aug}}$, which gives $\hat{\beta}_{\text{np}}$:

$$\begin{aligned} \hat{\beta} &= \beta^\star + \big(\Phi^\top \Phi\big)^{-1} \Phi^\top \Xi, \\ \hat{\beta}_{\text{np}} &= \beta^\star + \big(\Phi^\top \Phi\big)^{-1} \Phi^\top \big(\Xi - F^{\text{n}}_{\text{aug}}\big). \end{aligned} \tag{35}$$

With the projection, $\hat{\beta}$ does not depend on the network output, since $\Phi^\top \tilde{F}^{\text{n}}_{\text{aug}} = 0$ by (32). [from V-B, M-mail] The offset does not change the first-order estimate (35).[end] Without the projection, $\Phi^\top F^{\text{n}}_{\text{aug}}$ is still in the estimate $\hat{\beta}_{\text{np}}$ [8, Eq. (25)][Dirk]. The expression for $\hat{\beta} - \beta^\star$ has the same form as Györök et al. [8, Eq. (21)], which they derived assuming ~~an exact fit of noise-free data~~ the model exactly recovers the dynamics on $\mathcal{D}_N$[Dirk] [8, Assumption 6]. We obtain it by minimising the squared one-step error. The estimate equals $\beta^\star$ if and only if $\Phi^\top \Xi = 0$, ~~requiring Ξ to be~~ which means that Ξ is[Dirk] orthogonal to the columns of Φ on the tuples [8, Condition 4, Remark 5]. ~~This is a first-order result for the one-step least-squares problem on the fixed reference slice $\bar{x} = 0$; it does not predict the parameter bias from minimising the closed-loop loss (18) with active added states.~~ This holds for the one-step error, not for the closed-loop loss (18).[Dirk] [**Note:** Issue: $\hat{\beta}$ reads as the trained estimate of Table III, but it is never computed; as a number the first-order estimate fails (predicted bias 100 to 1300 %, observed a few

to 30 %). It only explains what the network overlap and the truth overlap in Table III measure. Question: keep it under another name, or replace it by a citation of [8, Eq. (21)]?]

D. Overlap with the parameter directions

We measure orthogonality using the normalised inner product, with the physical state normalised using (12):

$$\rho(Z) = \max_{Q \in \operatorname{range}\Phi \setminus \{0\}} \frac{\langle Q, Z\rangle}{\|Q\|_2 \, \|Z\|_2} = \frac{\|\Pi Z\|_2}{\|Z\|_2}, \qquad \Pi = \Phi\big(\Phi^\top \Phi\big)^{-1}\Phi^\top, \tag{36}$$

[**Note:** Changed (own): the $\cos\angle(a,b)$ and c_{neg} lines were removed from (36); the cosine is now written inside ρ. Changed (Dirk): d renamed Z (d is a physical parameter; Z is unused in the thesis); max variable v renamed Q (v_k is the measurement noise).] where ~~$a, b, d \in \mathbb{R}^{6N}$ are~~ $Z \in \mathbb{R}^{6N}$ is[own] nonzero and Π is the orthogonal projector onto $\operatorname{range}\Phi$. The overlap [Dirk] ~~$\rho(d)$~~ $\rho(Z)$ [Dirk] ~~gives~~ is[Dirk] the largest normalised inner product between ~~d~~ Z[Dirk] and a vector in $\operatorname{range}\Phi$. It is zero if and only if ~~$\Phi^\top d = 0$~~ $\Phi^\top Z = 0$[Dirk]. ~~Negation is measured by c_{neg}, which is the cosine between the network output and the one-step state change predicted by the baseline.~~[own]

VI. Experiment Design

- Opening: the records are simulated on a model of a confidential industrial gantry; quantities are given relative; design details and checks in Appendix C

In this section, we describe the simulated system, how the data is generated, the records, the compared models and the metrics.[own] The data used to validate our methods is simulated, with limits and noise level taken from a gantry of ASMPT. Its data is confidential, therefore positions are given relative to the training range, and velocities and accelerations relative to the machine limits. The design details ~~and their checks~~[own] are in Appendix C. ~~Section VI-A describes the simulated system, Section VI-B the closed loop and excitation, Section VI-C the records, Section VI-D the compared models and Section VI-E the metrics.~~[own]

A. Simulated system

- The data is simulated: the system is the baseline plus rail friction and a payload absorber, both absent from the baseline
- The friction subtracts from the actuator forces; a smoothed Coulomb law, Coulomb-like on the excited records
- The absorber adds one coordinate, coupled through the inertia; matrices written out as in Section II
- Parameter values with their basis; both absorber features lie above the crossover of the loop
- One known controller K_1, the same in every record; the injected force is the only feedforward
- Noise matched to the machine; noise-free twin of every record; SNR (?: confidentiality)

The system used to generate the data consists of the baseline model (4) with two additions. Fig. 6 shows the baseline model

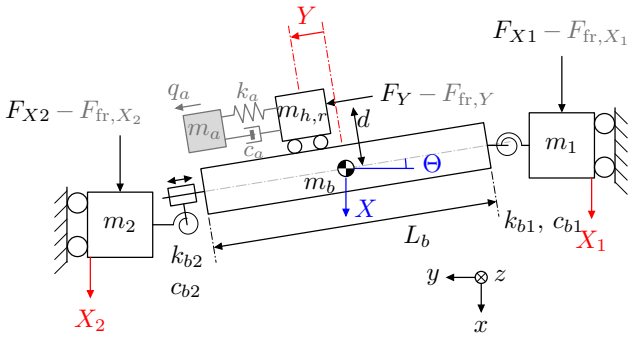


Fig. 6: Simulated system: the model of Fig. 1 with an added mass, spring and damper on the payload, and friction F_{fr} at the actuators. Grey elements represent the unmodelled dynamics of the baseline.

with both additions.[own] The first is Coulomb friction at the actuators X_1, X_2 and Y, as modelled by García-Herreros et al. [6]. We left it out of the baseline to add nonlinear unmodelled dynamics to the system, for the augmented model to capture. ~~The friction works as a force against each actuator force, this is modelled as a term entering (4) through the input.~~[own] For the second addition, we split the payload mass m_h into m_a and $m_{h,r} = m_h - m_a$, with m_a connected to $m_{h,r}$ by a spring and damper in the Y direction ~~(Fig. 6)~~[own]. This adds a vibration mode which the baseline model is not able to represent with its states. With the friction acting against each actuator force on the input, (4) becomes

$$M(Y)\ddot{q} + C\dot{q} + Kq = P\big(u_{\text{act}} - F_{\text{fr}}(P^\top \dot{q})\big), \qquad F_{\text{fr},i}(v_i) = c_{c,i}\tanh\big(v_i/v_{\text{fr}}\big), \tag{37a}$$

where v_i is the velocity of actuator $i \in \{X_1, X_2, Y\}$, with the actuator velocities given by $P^\top \dot{q}$, and v_{fr} sets the velocity scale of the smoothing. For $|v_i| \gg v_{\text{fr}}$, the friction approaches the Coulomb law $F_{\text{fr},i} = c_{c,i}\operatorname{sign}(v_i)$ of [6]. [from below, own] The friction levels c_c are taken from García-Herreros et al. [6, Table I].[end] ~~$v_{\text{fr}} = 1$ mm/s was chosen by simulating the loop with values between 0.1 and 10 mm/s. Higher values turn the friction into viscous damping under the multisine, and lower values keep the standstill servo error above the noise level.~~[moved to F-C, own]

The added mass adds the displacement q_a of m_a relative to $m_{h,r}$ along Y. We use the Lagrange-Euler equations of Section III-A, adding the kinetic energy of m_a, the elastic energy of k_a and the dissipation of c_a. As in the baseline, we leave out the Coriolis and centripetal terms. With $q_e = \begin{bmatrix} q^\top & q_a \end{bmatrix}^\top$, the system is

$$M_e(Y, q_a)\,\ddot{q}_e + C_e\dot{q}_e + K_e q_e = \begin{bmatrix} P\big(u_{\text{act}} - F_{\text{fr}}(P^\top\dot{q})\big) \\ 0 \end{bmatrix}, \tag{37b}$$

where M_e, C_e and K_e are given in Appendix C.1.[own] ~~where $M(Y)$ is (5a) with m_h the total payload mass. $M_a(Y, \ell)$ corrects it for m_a being at $Y + \ell$ instead of Y, where $\ell = L_0 + q_a$ and L_0 is the offset of the added mass along Y.~~[moved to F-A, own] ~~$k_a = m_a(2\pi f_a)^2$ and $c_a = 2\zeta_a\sqrt{k_a m_a}$,~~

~~with natural frequency f_a and damping ratio ζ_a.~~[moved to F-A, own] ~~The added mass changes the inertia through M_a, which also includes its coupling to Θ and Y.~~[own] We simulate the system with fixed-step RK4 at ~~f_s~~ f_{sim}[own]= 20 kHz. All parameter values are listed in Table VI. ~~The friction levels c_c are taken from García-Herreros et al. [?]Table I]garcia2013model.~~[moved to above, own] ~~The added mass creates an anti-resonance at f_a and a resonance at $f_a\sqrt{1+m_a/m_{h,r}} \approx 2.12\, f_b$, in the transfer function from F_Y to Y. Both are designed to be above the crossover of the loop with K_1 (Section VI-C). As a result, the feedback does not suppress them, and we can identify them from the data. All data is generated in closed loop with the controller K_1 of Appendix C.1; K_{fb} in (2) is K_1 discretised at T_s. The only feedforward, u^{ff} in (14), is the injected excitation force.~~[moved to VI-B, own] ~~Independent coloured noise is added to the position measurements of each axis, $y = q_{\text{act}} + v$. The simulated servo error matches at standstill the noise level and spectral shape of the industrial gantry, but its resonances are left out (Appendix C.1). The servo-error signal-to-noise ratio (SNR) ranges from 58 to 74 dB on the X actuators and 71 to 80 dB on Y, depending on the motion of the record. The data is also generated without measurement noise, with output y°.~~[moved to VI-B, own]

B. Closed loop and excitation

[from VI-A, own] All data is generated in closed loop with the controller K_1 of Appendix C.1[end], which has a bandwidth of $f_b = 100$ Hz[own from VI-A, own].[end] K_{fb} in (2) is K_1 discretised at T_s with the Tustin rule.[own] The feedforward u^{ff} in (14) is only the injected excitation force.[own] ~~Each record is 12 s long and uses one of five motion types: standstill at a fixed Y, slow sweeps of Y, X-Y Lissajous paths, point-to-point moves with S-curve profiles, and operational motion profiles. The motion profiles excite the low frequencies up to the loop crossover (Table IX).~~[moved to below, own]

[from VI-A, own] The added mass[end], spring and damper[own] ~~creates~~ create[own from VI-A, own] an anti-resonance at [end]~~f_a~~ 150 Hz[own from VI-A, own] and a resonance at [end]~~$f_a\sqrt{1+m_a/m_{h,r}} \approx 2.12\, f_b$~~ 212 Hz[own],[own from VI-A, own] in the[end] open-loop[own from VI-A, own] transfer function from F_Y to Y. We have chosen both frequencies to be above the crossover of the loop with K_1, so that the feedback does not suppress them in the data.[end]

[from above, own] Each record is 12 s long and uses one of five motion types: standstill at a fixed Y, slow sweeps of Y, X-Y Lissajous paths, point-to-point moves with S-curve profiles, and operational motion profiles. The motion profiles excite the low frequencies up to the loop crossover (Table IX)[end], and barely excite the added mass[own from above, own].[end] ~~The motion profiles barely excite the added mass, because of that a multisine force to each of the generalised channels X, Θ and Y is added. Except for the records of the operational motion profiles.~~ We therefore add a multisine force to each of the generalised channels X, Θ and Y, except in the records with an operational motion profile.[own] ~~The multisine band starts at the highest loop crossover, 106 Hz, because feedback suppresses the injected force below this frequency. The band ends at 297 Hz, covering 90 % of the difference between the frequency responses of the system and the baseline above the crossover. The band contains the anti-resonance and the resonance of the added mass for all training positions (Fig. 11).~~ The multisine band is excited from the highest crossover frequency of the loop with K_1 at 106 Hz up to 297 Hz (Fig. 11). Above the crossover, this band covers 90 % of the squared difference between the frequency responses of the system and the baseline.[own] Each record has its own random multisine phases, chosen for a low crest factor ~~(Appendix C.2)~~[own].[own]

[from VI-A, own] Independent coloured noise is added to the position measurements of each axis, $y = q_{\text{act}} + v$.[end] ~~The simulated servo error matches at standstill the noise level and spectral shape of the industrial gantry, but its resonances are left out~~ At standstill, the simulated servo error matches the noise level and spectral shape of the ASMPT gantry[own from VI-A, own] ~~(Appendix C.1)~~[own].[end] The noise is not modelled to capture the machine's resonances.[own] [from VI-A, own] The servo-error signal-to-noise ratio (SNR)[end] of the training records[own from VI-A, own] ranges from 58 to 74 dB on the X actuators and 71 to 80 dB on Y, depending on the motion profile[end] and the multisine[own from VI-A, own].[end] ~~The data is also generated without measurement noise, with output y°.~~[moved to VI-E, own]

~~The models run at~~ The records are downsampled to[own from VI-C, own] $T_s = 0.25$ ms.[end] The servo error is filtered by an eighth-order Butterworth low-pass at 1.6 kHz, we apply it forward and backward. The force is averaged over each sample.[Dirk] The simulated system and controller K_1 run at $f_{\text{sim}} = 20$ kHz, while Section II assumes the loop runs at T_s. We evaluate the effect of this difference in Table IX.[own]

C. Records

- A record: 12 s in closed loop with K_1, one of five motion types, a multisine added except on the operational moves; noisy and noise-free
- The motion profiles excite up to the crossover above the noise; the multisine band starts at the highest crossover, ends at 90 % of the model difference and holds the absorber features of the simulated system (simulation only)
- Split 18 / 6 / 11; validation only selects; results on all three sets, comparisons on the test records
- Test records by layer: interpolation, operational motion, extrapolation in one setting, stress record E6, multisine twins for evaluation only
- Model rate, anti-alias filter and window checked on the simulated loop (appendix)

~~The data is divided into five types of motion profiles and each record is 12 s (Appendix C.2). The motion profiles excite the low frequencies up till the loop crossover, for at least 59 dB above the noise. A multisine force is added to each of the generalised channels X, Θ, Y, except in records with an operational motion profile. The force is applied to the actuators through P^{-1}. The multisine band starts at the highest loop crossover $1.06\, f_b$, to prevent feedback~~

suppression. The band ends at $2.97\,f_b$, which keeps 90 % of the squared frequency-response error of the baseline above the crossover.[moved to VI-B, own] This was measured on the system without noise.[own] The band contains the anti-resonance and the resonance of the added mass (Fig. 11), for all the positions used in training.[moved to VI-B, own] This was determined from the frequency response of the known system model at each training position.[own]

We use 18 records for training, 6 for validation and 11 for testing (Table VIII in Appendix C.2). Seven of the test records without multisine are also evaluated with the training multisine added.[own] Training and validation include all five motion types, with . Their records stay within[own] $|Y| \leq Y_{\mathrm{tr}}$, up to[own] 75 % of the acceleration limit and 75 % of the velocity limit. The validation records are only [own]used to select the best-performing [own]checkpoints and the [own]hyperparameters for the black box[own].

We separate the test records into interpolation and extrapolation. Interpolation stays within the training range, while each extrapolation record exceeds at least [own]one of the limits on Y, acceleration or velocity. E5 exceeds multiple limits. Each extrapolation record has the same motion as one interpolation record, to show the effect of leaving the training range. We also simulate their motion with a multisine, because the motion profiles barely excite the added mass.[own] The results for all three sets are reported, but performance is evaluated on the test set. The models run at $T_s = 0.25$ ms[moved to VI-B, own] on training windows of 0.1 s[own]. The simulated system and its controller K_{I} run at $f_{\mathrm{sim}} = 20$ kHz, while Section II assumes the loop at T_s.[moved to VI-B, own] The data therefore meet these assumptions only approximately: K_{I} discretised at T_s differs from the feedback force of the system by a median of 0.5 % RMS, the force changes by a median of 10.4 % RMS within one hold of T_s, and the reduction of the records to T_s changes the noise-free output by at most $2.3 \cdot 10^{-9}$ m RMS (Table IX).[own] The rate, its anti-alias filter and the window length are supported by the checks in Table IX.[moved to VI-B, own]

D. Models

- Each comparison answers one aspect of the research question
- Grey-box models: same records, settings, budget and checkpoint rule; PS start of the added states; 10 % detuned start of the combinations; one seed
- The runs that justify $n_{\bar{x}}$, PS and the start are in the appendix (?: label)
- Black box: SUBNET of the closed loop, no physics, no controller, sized on the validation records

The models of Table I are compared to answer each aspect of Section I.

The models of Table I are compared in pairs, each answering one aspect of the research question. The pairs are the final model $\mathcal{M}_{\mathrm{OBC}}$ with the baseline $\mathcal{M}_{\mathrm{LPV}}$ and the black box $\mathcal{M}_{\mathrm{BB}}$ for the augmentation, and $\mathcal{M}_{\mathrm{OBC}}$ with $\mathcal{M}_{\mathrm{U}}$ for the orthogonal construction. The extrapolation records compare the final model with the black box (Aspect 4).[own] $\mathcal{M}_{\mathrm{LPV}}$ is the baseline at the true parameters.[own]

TABLE I: Models.

Model	Baseline	Learned states	OBC
$\mathcal{M}_{\mathrm{LPV}}$	true, not trained	0	no
$\mathcal{M}_{\mathrm{U}}$	trained	2	no
$\mathcal{M}_{\mathrm{OBC}}$	trained	2	yes
$\mathcal{M}_{\mathrm{BB}}$	none	16	no

We identify $\mathcal{M}_{\mathrm{U}}$ and $\mathcal{M}_{\mathrm{OBC}}$ by minimising the loss of Section IV-C with the same training setup (Table XI). This includes PS to initialise the added states (Section IV-E). The parameter combinations θ_{base} start of the detuned start are[own] 10 % larger or smaller in magnitude than their true values, and every run uses the same signs. Every model is trained once, with the same seed.[own] Appendix ?? shows the runs used to choose $n_{\bar{x}}$ and the PS initialisation of the added states.[own]

We compare the simulation error of $\mathcal{M}_{\mathrm{OBC}}$ and $\mathcal{M}_{\mathrm{LPV}}$, to test whether the augmentation is able to improve the performance by capturing the missing dynamics. To determine the number of added states and whether to use PS initialisation, we compare the simulation error of $\mathcal{M}_{\mathrm{U}}$ for each combination. These runs start from the true parameters, so that parameter errors do not affect the choice.[Dirk]

The simulation error of $\mathcal{M}_{\mathrm{OBC}}$ and $\mathcal{M}_{\mathrm{U}}$ determines whether the constraint on the neural network limits the performance of the model, and whether OBC keeps the parameters interpretable. Both are trained from the true parameters and from the 10 % detuned start, to test whether the estimate is unique depends on the start[own].[Dirk]

We test generalisation by comparing the simulation error of $\mathcal{M}_{\mathrm{OBC}}$, $\mathcal{M}_{\mathrm{LPV}}$ and $\mathcal{M}_{\mathrm{BB}}$ on the interpolation and extrapolation records.[Dirk]

The black box $\mathcal{M}_{\mathrm{BB}}$ is a SUBNET model [15] that predicts the closed-loop servo error from the reference and injected force, without modelling the controller separately (Appendix C.4). Its state count, network widths and learning rate are selected from 11 configurations using the validation records, with the same data and training and encoder window as the grey-box models.[own] It uses the same records, training window and encoder window as the grey-box models.[own]

E. Metrics

- Every model is simulated over whole test records: grey-box models in closed loop with K_1, the black box in free run
- Primary metric: the simulation error relative to the servo error, NRMS_e, per stage axis; beside it the RMS in metres and the noise floor
- Parameter error per combination against the true values; m_Δ scaled by the mean actuator mass; simulation only
- The two predictions of Section V: the network stays out of the parameter directions, and the remaining parameter error is the first-order bias

All models are simulated over the full records. The grey-box models use the closed loop of Section IV-B, while the black-box model uses the reference and injected force as inputs.

We compare the predicted servo error with the recorded servo error $y - r$, using the known reference. The main metric is the NRMS of [15, Eq. (43)], adapted by using the servo error as output. [**Note:** Issue: the research plan names the BFR as the primary measure, but on the position output every model scores almost 100 %, because the reference motion dominates the output. On the servo error the BFR equals $100-\mathrm{NRMS}_e$, so I report NRMS_e. Question: is this departure from the research plan acceptable?] from VI-B, own The data is also generated without measurement noise, with output y°.end ~~We also report the RMS in metres and the noise floor:~~ The NRMS is computed as own

$$\mathrm{RMS}_j = \Big(\frac{1}{N-n-1}\sum_{k=n+1}^{N-1}(\hat{y}_{j,k}-y_{j,k})^2\Big)^{1/2}, \qquad \mathrm{NRMS}_{e,j} = \frac{\mathrm{RMS}_j}{\sigma_{e,j}}, \tag{38}$$

where j is the stage axis, N the record length, $n+1$ the encoder window length and $\sigma_{e,j}$ the ~~population~~ own standard deviation of the servo error $y_j - r_j$ ~~over the same samples~~ for $k = n+1$ to $N-1$ own. An NRMS of 100 % means the model performs no better than predicting the mean servo error. The noise floor is taken as the NRMS_e of (38), with $\hat{y}$ replaced by the noise-free output y°. own ~~The noise floor is the RMS difference between the measured and noise-free outputs.~~ own

~~The parameter error of combination j is [display], where the true value of $m_\Delta = m_1 - m_2$ is close to zero, so we scale its error by the mean actuator mass $(m_1 + m_2)/2$ instead.~~ own

To compare the parameters of $\mathcal{M}_\mathrm{U}$ and $\mathcal{M}_\mathrm{OBC}$, own the relative error of the estimated combinations is computed against their true values, which are known in simulation. The overlap ρ of (36) shows whether the network stays out of the parameter directions. It is computed on the training, validation and interpolation test records. own ~~For the orthogonal construction of Section V, the overlap ρ of (36) is computed at $\bar{x} = 0$ on the one-step tuples of the training, validation and test records (test: I1 to I6), for the network of $\mathcal{M}_\mathrm{U}$, the corrected network of $\mathcal{M}_\mathrm{OBC}$ and the missing dynamics Ξ. On the training tuples the corrected overlap is zero by construction.~~ own

VII. Results and Discussion

- Test records, NRMS_e with the noise floor; training and validation in the appendix (D-269)

[Changes: black = original, blue = original, moved here (from … end), green = changed or new, ~~struck~~ = deleted ("moved to" = moved, not deleted), superscript = comment id.]

In this section, we analyse the models of Table I for each aspect of Section I. own All errors are reported on the test set in NRMS_e of (38), against the noise floor. Appendix B gives the training and validation records. ~~Sections VII-A to VII-C give the results for the three aspects of Section I in order, and Section VII-D discusses them.~~ own

A. Augmentation structure

- Final model against the baseline and the black box, with and without multisine (Table II, Fig. 7)

TABLE II: NRMS_e per axis on the test records, mean over the 10 records with a multisine and the 8 without. $\mathcal{M}_\mathrm{OBC}$ starts from the detuned parameters, the other grey-box models from the true parameters.

	Multisine			No multisine		
NRMS_e (%)	X_1	X_2	Y	X_1	X_2	Y
$\mathcal{M}_\mathrm{LPV}$	13.3	13.3	83.7	4.3	4.3	3.3
$n_{\bar{x}} = 0$	9.5	8.5	60.4	5.0	4.3	19.9
$n_{\bar{x}} = 2$, plain	8.0	7.3	53.6	4.3	4.2	13.4
$n_{\bar{x}} = 2$, PS	7.1	6.8	6.1	4.0	3.9	4.0
$n_{\bar{x}} = 8$, PS	6.7	6.1	6.6	3.9	3.9	4.0
$\mathcal{M}_\mathrm{OBC}$	7.7	7.1	6.2	4.5	4.5	3.7
Noise floor	0.05	0.04	0.01	0.04	0.04	0.01

- The final model has the resonance of the system, the baseline none (Fig. 9)
- PS start and added states compared on validation in the appendix

For Aspect 1, we compare the augmentation structures with the baseline ~~and the black box~~ own. Table II compares the final grey box model $\mathcal{M}_\mathrm{OBC}$ own with the baseline at the true parameters, ~~and with the black box~~ own. For the data with multisine, the final model lowers the error on Y by more than an order of magnitude and almost own halves the error on X_1 and X_2. For the data without multisine both models achieve similar results. ~~Fig. 7 shows this on record I1, and Fig. 9 shows that the final model is able to capture the resonance of the system, which is not modelled in the baseline.~~ The final model has a lower error than the baseline on record I1 (Fig. 7). The ~~augmented~~ final own model is able to capture the resonance of the system (Fig. 9). Dirk

Without added states or without PS, the error on Y with multisine stays above 50 %. Two added states with PS reach 6.1 % and eight added states gives no significant improvement. Fig. 8 shows that the models trained with PS have a significantly lower validation error, compared to the other models during training. own The missing dynamics are therefore only captured with added states and PS. own

[**Note:** Figuur heeft meerwaarde. om iets te laten zien niet de 3x3 frf.]

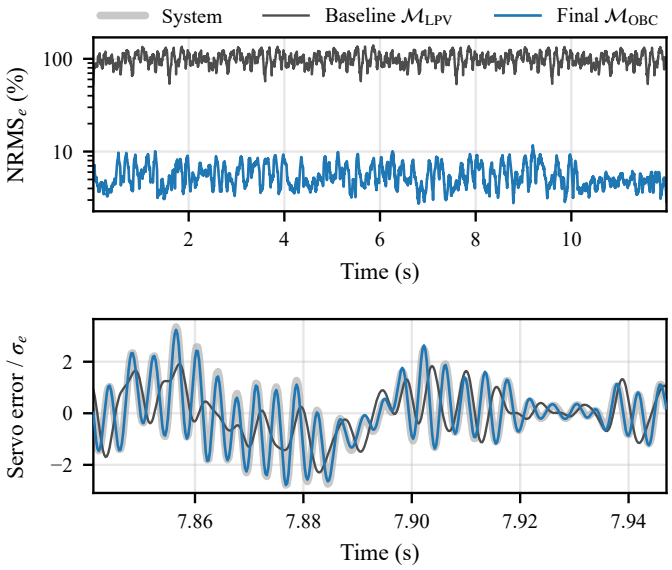


Fig. 7: Test record I1, Y axis. Top: NRMS_e of (38) over 50 ms. Bottom: servo error $y-r$ of the system and $\hat{y}-r$ of the models, divided by $\sigma_{e,Y}$.

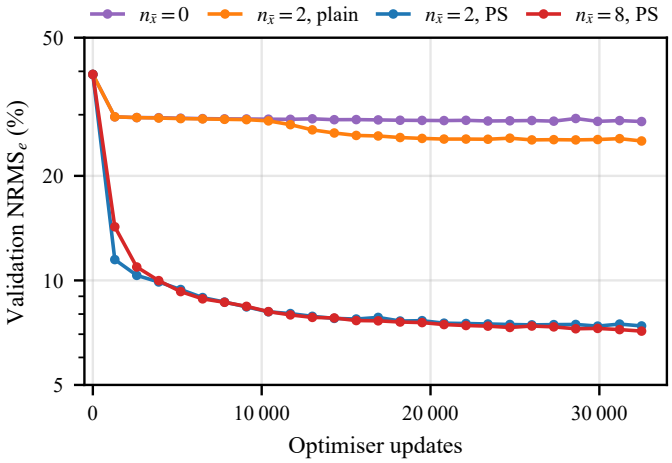


Fig. 8: Validation NRMS_e during training, mean over the validation records and axes. Update 0 equals the baseline.

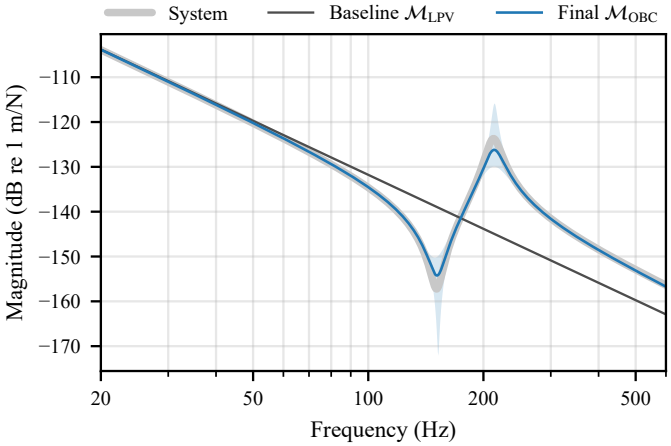


Fig. 9: Open-loop Bode magnitude from F_Y to Y at $Y=0$, from the models linearised at standstill. Shaded: final model over $|Y| \le 1.3\,Y_{\mathrm{tr}}$.

B. Interpretability

- Combination errors, full model, physical part and overlaps, $\mathcal{M}_\mathrm{U}$ against $\mathcal{M}_\mathrm{OBC}$ from the detuned start (Table III)
- Correct start and the models without added states in the appendix
- Uniqueness from the two starts (?: row requested, revnote at the end)

For Aspect 2, we compare the estimates of $\mathcal{M}_\mathrm{OBC}$ and $\mathcal{M}_\mathrm{U}$.[own] Table III ~~compares $\mathcal{M}_\mathrm{OBC}$ with $\mathcal{M}_\mathrm{U}$, both from the detuned start (Aspect 2)~~ gives the estimates from the detuned and the true start[own]. ~~Both models have deviated further from the true combinations, than their initial values.~~[moved to below, own] ~~$\mathcal{M}_\mathrm{U}$ deviates close to double that of $\mathcal{M}_\mathrm{OBC}$.~~[own] ~~The combinations describing the damping deviate the most, while the mass combinations stay close to their true values.~~[moved to below, own] The estimates of $\mathcal{M}_\mathrm{OBC}$ from the two starts differ by 31.3 %, while those of $\mathcal{M}_\mathrm{U}$ differ by 36.4 %.[own] The final model with the orthogonality constraint and the unconstrained model ~~perform equally well~~ have similar performance[Dirk]. OBC keeps the network out of the parameter directionson the training records[own]: its network overlap is much lower than that of $\mathcal{M}_\mathrm{U}$(Table IV)[own]. The network overlap of $\mathcal{M}_\mathrm{OBC}$ is also much lower than that of $\mathcal{M}_\mathrm{U}$ on the validation and test records. It increases from 0.00 on the training records to 0.24 on the test records.[Dirk] ~~The~~ However, the[own] missing dynamics are not orthogonal to these directions, their overlap is 0.59. ~~From the correct start, OBC also ends closer to the true values (Appendix ??).~~[own] ~~Without added states, $\mathcal{M}_\mathrm{U}$ ends closer instead.~~[moved to below, own] [from above, own] Both models have deviated further from the true combinations,[own] than their initial values.[end] With the detuned start, $\mathcal{M}_\mathrm{U}$ deviates 83.1 % from the true values and $\mathcal{M}_\mathrm{OBC}$ 69.1 %.[own] ~~Without added states, $\mathcal{M}_\mathrm{U}$ ends closer instead.~~[own] [from above, own] The parameter [Dirk]combinations describing the damping deviate the most, while the mass combinations stay close to their true values.[end] ~~OBC does not make the estimates unique, because they still depend on where training starts.~~ The estimates of both models still depend on where training starts.[own][own]

TABLE III: Relative error of the ten estimated combinations of $\mathcal{M}_\mathrm{U}$ and $\mathcal{M}_\mathrm{OBC}$, trained from the detuned and from the true start. Distances: Euclidean norm over the ten combinations.

		$\mathcal{M}_\mathrm{U}$		$\mathcal{M}_\mathrm{OBC}$	
Rel. error (%)	Detuned start	Detuned	True	Detuned	True
$k_{b,\Sigma}$	−10.0	−21.8	−7.6	−7.1	−9.7
c_{g1}	+10.0	+45.6	+27.7	+34.0	+23.6
c_{g2}	+10.0	+45.9	+27.2	+28.4	+16.2
c_y	+10.0	+43.3	+29.5	+47.4	+34.1
$c_{b,\Sigma}$	−10.0	+14.7	+29.2	+21.8	+35.7
m_h	−10.0	−1.3	+1.4	−2.0	+1.8
m_Σ	+10.0	+2.6	+1.3	+0.5	−0.6
$m_\Delta{}^*$	−0.5	−0.5	−0.1	−2.1	−1.6
J_eff	−10.0	−2.2	+4.1	−2.7	+4.7
d	−10.0	−11.7	−9.1	+3.7	−12.7
Distance to truth	30.0	83.1	58.3	69.1	59.5
Distance between starts		36.4		31.3	

* relative to the mean actuator mass, because its true value is close to zero.

TABLE IV: Overlaps of (36) and NRMS_e of $\mathcal{M}_\mathrm{U}$ and $\mathcal{M}_\mathrm{OBC}$, trained from the detuned start. The NRMS_e is the mean over the records with and without multisine.

	$\mathcal{M}_\mathrm{U}$	$\mathcal{M}_\mathrm{OBC}$
$\rho(F_\mathrm{aug})$, training records	0.85	0.00
$\rho(F_\mathrm{aug})$, validation records	0.84	0.08
$\rho(F_\mathrm{aug})$, interpolation test records	0.84	0.24
$\rho(\Xi)$, training records	0.51	
$\rho(\Xi)$, validation records	0.57	
$\rho(\Xi)$, interpolation test records	0.59	
NRMS_e (%), 10 records with a multisine		
X_1	7.1	7.7
X_2	6.5	7.1
Y	6.4	6.2
NRMS_e (%), 8 records without		
X_1	4.0	4.5
X_2	3.9	4.5
Y	4.0	3.7

C. Generalisation

- Final model, baseline and black box per test record, each E record beside its I record (Fig. V)
- Multisine twins of the records without multisine

For Aspect 3, we compare the final model with the baseline and the black box on the interpolation (I) and extrapolation (E) records (Table V).[own] ~~The error per test record are shown in Fig. V, for extrapolation (E) and interpolation (I) (Aspect 3).~~[own] The errors between E and I differ only by at most 0.5 points.[Dirk] Without multisine the model performs ~~as well or slightly above~~ as well as or slightly worse than[own] the baseline. ~~The black box has 74 to 193 % on the records without multisine, and it is below the final model only on I2 X_2.~~ The black box is worse than the final model with and without multisine, and reaches 97 to 128 % without it.[own] It even performs worse than the baseline model, except on Y with multisine.[Dirk] ~~On the multisine twins, the final model stays below 10 % on Y, while the baseline and the black box are above 47 %.~~ With multisine, the final

TABLE V: NRMS_e per axis on the test records, mean over the interpolation (I) and extrapolation (E) records with and without a multisine.

		Multisine		No multisine	
NRMS_e (%)		I	E	I	E
$\mathcal{M}_{\mathrm{LPV}}$	X_1	14.3	12.3	4.5	4.1
	X_2	14.6	12.0	4.6	4.1
	Y	89.5	78.0	3.4	3.1
$\mathcal{M}_{\mathrm{OBC}}$	X_1	7.9	7.5	4.7	4.3
	X_2	7.3	6.8	4.7	4.3
	Y	6.1	6.3	3.8	3.5
$\mathcal{M}_{\mathrm{BB}}$	X_1	78.5	82.4	104.8	108.5
	X_2	79.2	81.1	97.0	99.8
	Y	65.9	74.7	127.7	127.0

model stays below 8 % on all axes, while the errors of the baseline and the black box exceed 65 % on Y.[own] The final model generalises to the held-out positions and unseen motion profiles, but only improves on the baseline when the multisine excites the missing dynamics.[own]

D. Discussion

The improvement on ~~the multisine records~~ Y with multisine[own] is mainly caused by the learned ~~absorber mode~~ vibration mode of the added mass[own](Fig. 9)[own]. The static correction without added states, is only able to lower the error on Y from 83.7 % for the baseline to 60.4 %, while two added states with PS reach 6.1 % (Table II). Training without PS does not seem to find this mode in the current setting, it stays above 50 %.[own] Without multisine, the final model performs slightly worse than the baseline. For these records the baseline error is only a few percent of the serve error, and only four of the eighteen training records are without multisine. Resulting in the records without multisine contributing relatively little to the loss. The final model remains far above the noise floor on every record. A possible reason is that the relative error is already small, so reducing it further requires a better tuned optimisation. The augmentation might not be able to capture the unmodelled friction, or the difference in controller rates contributes to this. From these results, we cannot conclude what the reason is.

OBC is able to keep the neural network out of the parameter directions on the training tuples, and for the most part on the validation and test records compared to $\mathcal{M}_{\mathrm{U}}$ (Table IV). However, the missing dynamics are not orthogonal to these directions, their overlap is 0.59 (Table IV). Orthogonality is defined to hold only to first order and at $\bar{x} = 0$, and the added states enter through f_{aug} without projection. This holds for a single step, not for the simulated output, where the model is simulated over multiple steps with nonzero added states. Even when training starts from the true parameters, the damper combinations deviate the most, while the mass combinations stay close to their true values (Table III). A possible explanation is that the dampers absorb part of the friction, since both produce a force in the opposite direction of the velocity. For both models, the estimates from the two starts differ by a similar amount (Table III), so the trained estimate still depends on the start. Given the current results, the estimates cannot be used to trace a change in the machine back to a physical parameter.

For the interpolation and extrapolation records both the baseline and the final model have a similar error (Table V), However, most extrapolation records extend beyond the training range in only one direction. The black box model underperforms, while Hoekstra et al. report similar results for black-box and dynamic augmented models [5, Table 8]. Possible reasons for the underperformance are that it starts from a random initialisation, instead of for example a best linear approximation. The black box model also has to learn the controller dynamics, because the controller is not included in the training loop. What we do see is that the black box model performs much better on the records with multisine than on the records without (Table VII). For the records without multisine, the reference is the only input. The black box model therefore has to learn the complete closed-loop response, while the grey box model uses the known controller and starts from the baseline.

The results are limited to simulation with one seed and the same controller used to generate the data. Training and evaluation are both performed in closed loop, where the controller can suppress errors in the plant model. A different controller might be able to reveal these errors. The recorded input is correlated with the measurement noise through the feedback, while SUBNET assumes they are independent [15]. Therefore, we have not shown the consistency of the estimate.

For future work, the model should be evaluated on measured data, and evaluated with a different controller. OBC could also be extended to the simulated output, using its Jacobian over a training window instead of the Jacobian of a single state update.

[**TODO:** Missing: the numbers of $\mathcal{M}_{\mathrm{OBC}}$ (R1) in this section are preliminary (update 19500 of 33400). Candidate: update them after the final R1/R2 rerun (basis: handoff 2026-10-18, Sec. 3).] [**TODO:** Missing: the network overlap of $\mathcal{M}_{\mathrm{OBC}}$ in Table III comes from update 18200 while labelled 19500, so the text gives no number for it. Candidate: recheck after the final R1 rerun (basis: handoff 2026-10-18, Sec. 12).] [**TODO:** Missing: the caption of Fig. V says "RMS, mean over X_1, X_2, Y", while the text reads NRMS_e per axis. Candidate: check the figure and update the caption (Jasper p9: say error of what with respect to what).] [**TODO:** Are state Gramians useful for explaining each added state's contribution, or would intervention-based ablation be more direct and less linearisation-dependent?]

VIII. Conclusion

- Discussion per aspect: explanation, limit and test of each result; then the limits and departures that bound all three (handoff 2026-10-18)
- Conclusion: the answer per aspect and to the research question, the central limit, one recommendation

[Changes: black = original, blue = original, moved here (from … end), green = changed or new, ~~struck~~ = deleted ("moved to" = moved, not deleted), superscript = comment id.]

~~The improvement on the multisine records comes mainly from the learned absorber mode (Fig. 9). The results in Table ?? (Appendix ??) show that both the added states and their PS initialisation are needed to obtain this improvement. Without multisine, the final model performs slightly worse than the baseline. The baseline is already accurate on these records, so the learned correction might overfit on the multisine records and introduce small errors on the records without multisine.~~[moved to VII-D, M-p15] ~~OBC reduces the final combination error compared with the unconstrained model (47.7 against 83.1 %), but the parameters still end up further from the truth than at their initial values (Table III). The missing dynamics have a truth overlap of 0.59 with the parameter directions. The truth overlap of 0.59 means $\Phi^\top \Xi \neq 0$, so the recovery condition of Section V-C is not met and OBC is not expected to recover the true values. The orthogonality holds only to first order and at $\bar{x} = 0$. During training, the added states enter the physical states through f_{aug} without projection. This is a limitation of imposing the constraint at $\bar{x} = 0$. With OBC the estimates from the true and detuned starts are closer together, but still differ (Table III). The results therefore show less dependence on initialisation, but does not establish uniqueness.~~[moved to VII-D, M-p15] ~~The black box model performs poorly on its training records and had not converged within the available budget (Table X). The black box model is also randomly initialised and not trained with the known controller. Having a disadvantages starting position. t~~[moved to VII-D, M-p15] ~~The results are limited to simulation with one seed and the same controller used to generate the data. Training and evaluation are both performed in closed loop, where the controller can suppress errors in the plant model. It remains unclear whether the improvements would hold with a different controller or on measured data.~~[moved to VII-D, M-p15]

For the dual-drive gantry, a parallel augmentation with added states is able to capture the vibration mode of the added mass, spring and damper when it is trained in closed loop with the known controller and the added states are initialised with PS. The resulting model reduces the prediction error on Y when this mode is sufficiently excited in the data. The final model performs slightly worse than the baseline model on records without multisine excitation. The model generalises to held-out positions and unseen motion profiles: the prediction errors on the interpolation and extrapolation records are similar, and the final model performs better than the black box model. Compared to unconstrained estimation, OBC is able to reduce the overlap between the neural network and the parameter directions. However, the parameter estimates still deviate from their true values and depend on where training starts. In conclusion, a parallel augmentation with added states initialised with PS improves the prediction of the gantry, but the uniqueness provided by OBC for a single state update does not make the trained parameters interpretable as defined in Section I. These results are limited to simulation with one controller.

REFERENCES

[1] J. Hwang and S. D. Noh, "Digital twin-based optimization of operational parameters for cluster tools in semiconductor manufacturing," *IEEE Access*, vol. 12, pp. 122 078–122 100, 2024.

[2] International Roadmap for Devices and Systems (IRDS), "International roadmap for devices and systems: 2022 edition executive summary," IEEE, Tech. Rep., 2022.

[3] T. Workman, L. Mirkarimi, J. Theil, G. Fountain, K. M. Bang, B. Lee, C. Uzoh, D. Suwito, G. Gao, and P. Mrozek, "Die to wafer hybrid bonding and fine pitch considerations," in *2021 IEEE 71st Electronic Components and Technology Conference (ECTC)*. IEEE, 2021, pp. 2071–2077.

[4] B. M. Kessels, "Ai-based model updating for nonlinear dynamical systems," PhD thesis, Eindhoven University of Technology, Eindhoven, The Netherlands, 2025.

[5] J. H. Hoekstra, B. M. Györök, R. Tóth, and M. Schoukens, "Learning-based augmentation of first-principle models: A linear fractional representation-based approach," 2026, submitted to Automatica.

[6] I. García-Herreros, X. Kestelyn, J. Gomand, R. Coleman, and P.-J. Barre, "Model-based decoupling control method for dual-drive gantry stages: A case study with experimental validations," *Control Engineering Practice*, vol. 21, pp. 298–307, 2013.

[7] C. S. Teo, K. K. Tan, S. Y. Lim, S. Huang, and E. B. Tay, "Dynamic modeling and adaptive control of a h-type gantry stage," *Mechatronics*, vol. 17, pp. 361–367, 2007.

[8] B. M. Györök, M. Schoukens, T. Péni, and R. Tóth, "Orthogonal-by-construction augmentation of physics-based input-output models," *IFAC Journal of Systems and Control*, vol. 35, p. 100376, 2026.

[9] R. Isermann, "Fault diagnosis of machines via parameter estimation and knowledge processing—tutorial paper," *Automatica*, vol. 29, no. 4, pp. 815–835, 1993.

[10] J. Willard, X. Jia, S. Xu, M. Steinbach, and V. Kumar, "Integrating scientific knowledge with machine learning for engineering and environmental systems," *ACM Computing Surveys*, vol. 55, no. 4, Nov. 2022.

[11] G. E. Karniadakis, I. G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang, and L. Yang, "Physics-informed machine learning," *Nature Reviews Physics*, vol. 3, no. 6, pp. 422–440, 2021.

[12] M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *Journal of Computational Physics*, vol. 378, pp. 686–707, 2019.

[13] J. P. Noël and J. Schoukens, "Grey-box state-space identification of nonlinear mechanical vibrations," *International Journal of Control*, vol. 91, no. 5, pp. 1118–1139, 2018.

[14] A. Retzler, R. Tóth, M. Schoukens, G. I. Beintema, J. Weigand, J.-P. Noël, Z. Kollár, and J. Swevers, "Learning-based augmentation of physics-based models: an industrial robot use case," *Data-Centric Engineering*, vol. 5, p. e12, 2024.

[15] G. I. Beintema, M. Schoukens, and R. Tóth, "Deep subspace encoders for nonlinear system identification," *Automatica*, vol. 156, p. 111210, 2023.

[16] R. Tóth, *Modeling and Identification of Linear Parameter-Varying Systems*, ser. Lecture Notes in Control and Information Sciences. Berlin, Heidelberg: Springer, 2010, vol. 403.

[17] R. Drenth, J. H. Hoekstra, M. Schoukens, and R. Tóth, "Efficient learning of affine and rational dependency LPV models with linear fractional representation," 2026.

[18] M. Gautier and W. Khalil, "Direct calculation of minimum set of inertial parameters of serial robots," *IEEE Transactions on Robotics and Automation*, vol. 6, no. 3, pp. 368–373, 1990.

[19] C. M. Bishop, *Neural Networks for Pattern Recognition*. Oxford: Clarendon Press, 1995.

[20] U. Forssell and L. Ljung, "Closed-loop identification revisited," *Automatica*, vol. 35, no. 7, pp. 1215–1241, 1999.

[21] J. H. Hoekstra, B. M. Györök, R. Tóth, and M. Schoukens, "Encoder initialisation methods in the model augmentation setting," 2026.

[22] A. Hefny, C. Downey, and G. J. Gordon, "Supervised learning for dynamical system learning," in *Advances in Neural Information Processing Systems*, vol. 28, 2015.

[23] C. Downey, A. Hefny, B. Li, B. Boots, and G. Gordon, "Predictive state recurrent neural networks," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
[24] A. Venkatraman, N. Rhinehart, W. Sun, L. Pinto, M. Hebert, B. Boots, K. M. Kitani, and J. A. Bagnell, "Predictive-state decoders: Encoding the future into recurrent networks," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
[25] B. M. Györök, J. H. Hoekstra, J. Kon, T. Péni, M. Schoukens, and R. Tóth, "Orthogonal projection-based regularization for efficient model augmentation," in *Proceedings of the 7th Annual Conference on Learning for Dynamics and Control*, ser. Proceedings of Machine Learning Research, N. Ozay, L. Balzano, D. Panagou, and A. Abate, Eds., vol. 283, 2025.

APPENDIX A
SUPPORTING DERIVATIONS

A.1 Nominal physical parameters

Table VI lists the nominal values used by the baseline. These values are from the dual-drive gantry of García-Herreros et al. [6, Table I]. The lower block shows the parameters of the simulated system, which are omitted from the baseline (Section VI-A).

TABLE VI: Baseline parameters (upper block) and the additional Coulomb friction and absorber parameters of the simulated system (lower block).

Parameter	Value	Parameter	Value
m_1	10.2 kg	m_2	10.7 kg
m_b	22.8 kg	m_h	10.1 kg
J_b	1.00 kg m^2	J_h	0.05 kg m^2
c_{g1}	14.5 N s/m	c_{g2}	20.3 N s/m
c_y	10.0 N s/m	d	0.10 m
c_{b1}	9.0 N m s/rad	c_{b2}	9.0 N m s/rad
k_{b1}	1987.5 N m/rad	k_{b2}	1987.5 N m/rad
L_b	0.725 m		
c_{c,X_1}	16.80 N	c_{c,X_2}	18.35 N
$c_{c,Y}$	11.60 N	v_{fr}	1 mm/s
m_a	0.5 m_h	L_0	0.10 m
f_a	150 Hz	ζ_a	0.03

A.2 LPV-LFR realisation of the baseline

This appendix derives an exact LPV-LFR realisation of the baseline (7) and shows that it is well posed whenever $M(Y)$ is nonsingular.[Dirk] In (6), M_0 is ~~(5a) evaluated at $Y=0$~~ the part of (5a) without Y[Dirk], and

$$M_1 = \begin{bmatrix} 0 & -m_h & 0 \\ -m_h & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \qquad M_2 = \begin{bmatrix} 0 & 0 & 0 \\ 0 & m_h & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

[from III-B, Dirk] ~~A LFR representation~~ An LFR[own] is introduced to separate ~~this~~ the[Dirk] rational scheduling dependence of (7)[Dirk], with ~~an LTI part~~ a linear time-invariant (LTI) part[own] whose matrices do not depend on Y. We move the dependence out of the LTI part G and into a Delta-block $\Delta(Y)$, with the scheduling being connected in feedback with G (Fig. 10) [5], [17].[end] [from III-B, Dirk] Substituting (6) into (4) gives[end]

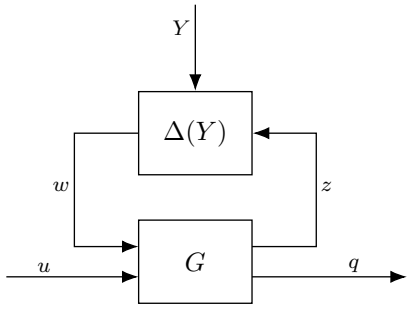


Fig. 10: Self-scheduled baseline interconnection. The Delta-block closes the latent variables of G through $w = \Delta(Y)z$, with $\Delta(Y) = YI_6$ and Y a component of the state.

$$M_0\ddot{q} + YM_1\ddot{q} + Y^2M_2\ddot{q} = u - C\dot{q} - Kq. \tag{39}$$

[from III-B, Dirk] Here Y only enters through $Y\ddot{q}$ and $Y^2\ddot{q}$, with $Y^2\ddot{q} = Y(Y\ddot{q})$. This lets us apply the same scheduling variable twice instead of introducing a second scheduling variable Y^2,[end]

$$\ddot{q} \xrightarrow{Y} a_1 \xrightarrow{Y} a_2, \tag{40}$$

[from III-B, Dirk] with $a_1 = Y\ddot{q}$ and $a_2 = Y^2\ddot{q}$. We treat a_1 and a_2 as independent signals, so (39) becomes[end]

$$M_0\ddot{q} + M_1a_1 + M_2a_2 = u - C\dot{q} - Kq, \tag{41}$$

[from III-B, Dirk] which contains no Y. This is allowed, because the Delta-block below imposes $a_1 = Y\ddot{q}$ and $a_2 = Ya_1$ again. The latent variables are $z = [\ddot{q}^\top \; a_1^\top]^\top$ and $w = [a_1^\top \; a_2^\top]^\top$, with the Delta-block mapping z to w,[end]

$$w = \begin{bmatrix} a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} Y\ddot{q} \\ Ya_1 \end{bmatrix} = \underbrace{\begin{bmatrix} YI_3 & 0 \\ 0 & YI_3 \end{bmatrix}}_{\Delta(Y)=YI_6} \begin{bmatrix} \ddot{q} \\ a_1 \end{bmatrix} = \Delta(Y)z. \tag{42}$$

[from III-B, Dirk] We write the LTI part G of the LFR in continuous time, adapting the discrete-time block structure of [17, Eq. (2)]:[end] [**TODO:** Check this]

$$\begin{bmatrix} \dot{\tilde{x}} \\ z \\ q \end{bmatrix} = G \begin{bmatrix} \tilde{x} \\ w \\ u \end{bmatrix}, \tag{43}$$

[from III-B, Dirk] where $\tilde{x} = [q^\top \; \dot{q}^\top]^\top$ is the state and q the output. The first block row is the state equation, with $\ddot{q}$ obtained from (41) by multiplying with M_0^{-1}.[end] [from III-B, Dirk] Conversely, closing the algebraic loop with (42) results in $a_1 = Y\ddot{q}$ and $a_2 = Y^2\ddot{q}$, which turns (41) back into (4). The LFR is therefore an exact realisation of the baseline. The LFR is well posed if the algebraic loop has a unique solution for every Y [17, Def. 1].[end]

~~With a_1 and a_2 from (40) treated as independent signals, solving $M_0\ddot{q} + M_1a_1 + M_2a_2 = u - C\dot{q} - Kq$ for $\ddot{q}$ with $R = M_0^{-1}$ gives $\ddot{q} = R(u - Kq - C\dot{q} - M_1a_1 - M_2a_2)$. With $\dot{\tilde{x}} = [\dot{q}^\top \; \ddot{q}^\top]^\top$, $z = [\ddot{q}^\top \; a_1^\top]^\top$, and $w = [a_1^\top \; a_2^\top]^\top$, the LTI part G of (43), using a continuous-time adaptation of the block partition in [?]Eq. (2)]drenth2026lpvlfr, is[own]~~ With $R = M_0^{-1}$ ($M_0 = M(0) \succ 0$, Appendix A.3), (41) gives $\ddot{q} = R(u - Kq - C\dot{q} - M_1a_1 - M_2a_2)$. Partitioning G in (43) as in [17, Eq. (2)], with rows $(\dot{\tilde{x}}, z, q)$ and columns $(\tilde{x}, w, u)$, gives the blocks[own] ~~with blocks[own]~~

$$\begin{aligned} A_x &= \begin{bmatrix} 0 & I_3 \\ -RK & -RC \end{bmatrix}, & B_w &= \begin{bmatrix} 0 & 0 \\ -RM_1 & -RM_2 \end{bmatrix}, & B_u &= \begin{bmatrix} 0 \\ R \end{bmatrix}, \\ C_z &= \begin{bmatrix} -RK & -RC \\ 0 & 0 \end{bmatrix}, & D_{zw} &= \begin{bmatrix} -RM_1 & -RM_2 \\ I_3 & 0 \end{bmatrix}, & D_{zu} &= \begin{bmatrix} R \\ 0 \end{bmatrix}, \\ C_y &= \begin{bmatrix} I_3 & 0 \end{bmatrix}, & D_{yw} &= 0, & D_{yu} &= 0. \end{aligned} \tag{44}$$

Here $\tilde{x}, z, w \in \mathbb{R}^6$ and $u, q \in \mathbb{R}^3$, so $G \in \mathbb{R}^{15\times 15}$. ~~The actuator frame enters only through $u = Pu_{\mathrm{act}}$ and $q_{\mathrm{act}} = P^\top q$ in (3).~~[own]

Substituting $w = \Delta(Y)z$ ~~into the z equation of (??)~~[own] gives the ~~algebraic~~[own] loop equation

$(I_6 - D_{zw}\Delta(Y))z = C_z x + D_{zu}u$. Written out, $(I_6 - D_{zw}\Delta(Y))z = C_z\tilde{x} + D_{zu}u$, with[own]

$$I_6 - D_{zw}\Delta(Y) = \begin{bmatrix} I_3 + YRM_1 & YRM_2 \\ -YI_3 & I_3 \end{bmatrix}.$$

The second block row gives $a_1 = Y\ddot{q}$, and the first then gives $M(Y)\ddot{q} = u - C\dot{q} - Kq$, that is (4), after multiplication by M_0. The condition of [?]Def. 1]drenth2026lpvlfr, that $I_6 - D_{zw}\Delta(Y)$ is nonsingular, is therefore the condition that $M(Y)$ is nonsingular: the Schur complement of the lower identity block gives[own] The Schur complement of the lower identity block gives[own]

$$\begin{aligned}\det\big(I_6 - D_{zw}\Delta(Y)\big) &= \det\big(I_3 + YRM_1 + Y^2RM_2\big) \\ &= \det M(Y)/\det M_0,\end{aligned}$$

so the condition of [17, Def. 1] holds if and only if $M(Y)$ is nonsingular.[own] Since $M(Y) \succ 0$ (Appendix A.3), the loop has the closed-form solution $\ddot{q} = M(Y)^{-1}(u - C\dot{q} - Kq)$ of (7), with $M(Y)^{-1} = \operatorname{adj} M(Y)/\det M(Y)$.[own] where $N_0 + YN_1 + Y^2N_2 = \operatorname{adj} M(Y)$ and $d(Y)$ are both quadratic in Y. The implementation evaluates the numerator with Horner's scheme and $d(Y)$ term by term.[own]

A.3 Positive definiteness of the inertia matrix

With $a = L_b m_\Delta/2$, (5a) in terms of (8) is

$$M(Y) = \begin{bmatrix} m_\Sigma + m_h & a - m_hY & 0 \\ a - m_hY & J_{\text{eff}} + m_h(d^2 + Y^2) & -m_hd \\ 0 & -m_hd & m_h \end{bmatrix}.$$

By Sylvester's criterion, applied to the principal minors from the lower-right corner, $M(Y) \succ 0$ if and only if

$$m_h > 0, \qquad m_h\big(J_{\text{eff}} + m_hY^2\big) > 0, \qquad \det M(Y) > 0,$$

with

$$\begin{aligned}\frac{\det M(Y)}{m_h} &= (m_\Sigma + m_h)(J_{\text{eff}} + m_hY^2) - (a - m_hY)^2 \\ &= m_\Sigma m_h\Big(Y + \frac{a}{m_\Sigma}\Big)^2 + (m_\Sigma + m_h)\Big(J_{\text{eff}} - \frac{a^2}{m_\Sigma}\Big).\end{aligned}$$

For $m_h, m_\Sigma, J_{\text{eff}} > 0$, this expression is positive for every Y if and only if (9) holds. Substituting the definitions of m_Σ, m_Δ and J_{eff} gives

$$\begin{aligned}m_\Sigma J_{\text{eff}} - \tfrac{L_b^2}{4}m_\Delta^2 &= m_\Sigma(J_b + J_h) \\ &\quad + \tfrac{L_b^2}{4}\big[4m_1m_2 + m_b(m_1 + m_2)\big] > 0.\end{aligned}$$

In (10), with $b = \gamma(2/L_b)\sqrt{m_\Sigma J_{\text{eff}}}$ and b_0 its value at θ_{base}^0,

$$r_0 = \frac{m_{\Delta,0}}{b_0}, \quad |r_0| < 1, \quad \eta_0 = \operatorname{atanh}(r_0), \quad s = \frac{m_{\Delta,0}}{b_0(1 - r_0^2)},$$

so that at $\beta = 0$, $m_\Delta = m_{\Delta,0}$ and $\partial m_\Delta/\partial\beta_{m_\Delta} = m_{\Delta,0}$.

TABLE VII: NRMS_e per axis on the training, validation and test records, mean over the records of each set with and without multisine.

	Multisine			No multisine		
NRMS_e (%)	X_1	X_2	Y	X_1	X_2	Y
Training records (14 with, 4 without multisine)						
$\mathcal{M}_{\text{LPV}}$	21.9	24.5	120.0	4.6	5.0	3.7
$\mathcal{M}_{\text{U}}$	9.4	10.2	6.6	4.1	4.3	4.3
$\mathcal{M}_{\text{OBC}}$	10.2	10.8	6.3	4.5	4.8	4.2
$\mathcal{M}_{\text{BB}}$	32.4	34.5	25.2	75.6	76.9	104.4
Validation records (4 with, 2 without multisine)						
$\mathcal{M}_{\text{LPV}}$	23.0	24.9	121.8	4.7	4.9	3.7
$\mathcal{M}_{\text{U}}$	9.9	10.2	6.6	4.1	4.2	4.3
$\mathcal{M}_{\text{OBC}}$	10.8	10.8	6.5	4.5	4.8	4.2
$\mathcal{M}_{\text{BB}}$	31.6	33.6	23.8	75.6	76.9	103.3
Test records (10 with, 8 without multisine)						
$\mathcal{M}_{\text{LPV}}$	13.3	13.3	83.7	4.3	4.3	3.3
$\mathcal{M}_{\text{U}}$	7.1	6.5	6.4	4.0	3.9	4.0
$\mathcal{M}_{\text{OBC}}$	7.7	7.1	6.2	4.5	4.5	3.7
$\mathcal{M}_{\text{BB}}$	80.5	80.2	70.3	106.7	98.4	127.4

A.4 Identifiable combinations

The matrices M_0, M_1, M_2, C and K depend on the combinations in θ_{base} as follows:

$$\begin{aligned} M_{0,11} &= m_\Sigma + m_h, & M_{0,12} &= \tfrac{L_b}{2}m_\Delta, \\ M_{0,22} &= J_{\text{eff}} + m_hd^2, & M_{0,23} &= -m_hd, \\ C_{11} &= c_{g1} + c_{g2}, & C_{33} &= c_y, \\ C_{12} &= \tfrac{L_b}{2}(c_{g1} - c_{g2}), & K_{22} &= k_{b,\Sigma}, \end{aligned}$$
$$M_{0,33} = -M_{1,12} = M_{2,22} = m_h,$$
$$C_{22} = c_{b,\Sigma} + \tfrac{L_b^2}{4}(c_{g1} + c_{g2}).$$

where $M_{i,jk}$ denotes entry (j,k) of M_i. For $m_h > 0$, we can recover these combinations from the matrix entries:

$$\begin{aligned} m_h &= M_{0,33}, & d &= -M_{0,23}/m_h, \\ m_\Sigma &= M_{0,11} - m_h, & m_\Delta &= 2M_{0,12}/L_b, \\ J_{\text{eff}} &= M_{0,22} - m_hd^2, & k_{b,\Sigma} &= K_{22}, \\ c_{g1} &= \tfrac{1}{2}C_{11} + C_{12}/L_b, & c_{g2} &= \tfrac{1}{2}C_{11} - C_{12}/L_b, \\ c_{b,\Sigma} &= C_{22} - \tfrac{L_b^2}{4}C_{11}, & c_y &= C_{33}. \end{aligned}$$

Therefore, two different values of θ_{base} cannot give the same matrices.

APPENDIX B
TRAINING, VALIDATION AND TEST RESULTS

Table VII gives the models of Section VII on the training, validation and test records.

APPENDIX C
EXPERIMENT DESIGN DETAILS

This appendix gives the details of Section VI and the checks of its design choices. The checks use the known simulated system where the text says so.[own]

C.1 Data-generating system

- The controller K_1 of every record
- The noise spectrum, fitted to the standstill servo error of the machine (?: confidentiality)
- The definition of the signal-to-noise ratio

The matrices of (37b) are[own]

$$M_{\mathrm{e}}(Y, q_a) = \begin{bmatrix} M(Y) + M_a(Y,\ell) & m_a \begin{bmatrix} 0 & -d & 1 \end{bmatrix}^{\top} \\ m_a \begin{bmatrix} 0 & -d & 1 \end{bmatrix} & m_a \end{bmatrix},$$

$$M_a(Y,\ell) = m_a \ell \begin{bmatrix} 0 & -1 & 0 \\ -1 & 2Y + \ell & 0 \\ 0 & 0 & 0 \end{bmatrix},$$

$$C_{\mathrm{e}} = \mathrm{diag}(C, c_a), \quad K_{\mathrm{e}} = \mathrm{diag}(K, k_a),$$

[from VI-A, own] where $M(Y)$ is (5a) with m_h the total payload mass. ~~$M_a(Y,\ell)$ corrects it for m_a being at $Y + \ell$ instead of Y, where~~ ,[own] $\ell = L_0 + q_a$ and L_0 is the offset of the added mass along Y.[end] [from VI-A, own] $k_a = m_a(2\pi f_a)^2$ and $c_a = 2\zeta_a\sqrt{k_a m_a}$, with natural frequency f_a and damping ratio ζ_a.[end]

The controller of every record is, per stage axis j,

$$K_{1,j}(s) = \kappa_j \, \frac{s + \omega_b/6}{s} \, \frac{s + \omega_b/3}{s + 3\omega_b} \, \frac{10\,\omega_b}{s + 10\,\omega_b}, \qquad (45)$$

where $\omega_b = 2\pi f_b$ is the bandwidth and κ_j sets the open-loop gain of axis j to one at ω_b, ~~for the diagonal entry of the rigid baseline at $Y = 0$ from stage force to stage position~~[own]. K_1 is discretised by the Tustin rule at ~~f_{s}~~ f_{sim}[own] for the data, and by the same rule at T_s for K_{fb} in the model[own].

~~The noise v_j has the one-sided spectrum~~[own] ~~where H_j is a second-order Butterworth low-pass with cutoff $f_{v,j}$, λ_j its level and $\sigma_{v,j}$ a white floor.~~[own] ~~λ_j and $f_{v,j}$ are fitted so that the simulated standstill servo error at $Y = 0$ matches the measured one of the industrial machine, and $\sigma_{v,j}$ is its level above 5 kHz.~~[own] The signal-to-noise ratio of Section VI-A is $\mathrm{SNR}_j = 20\log_{10}\big(\mathrm{RMS}(y_j^{\circ} - r_j)/\mathrm{RMS}(y_j - y_j^{\circ})\big)$ over the training records.

C.2 Excitation and records

- The multisine: grid from the width of the anti-resonance, best crest factor of 30 draws, amplitudes, no record needed scaling
- The upper band edge and the band over the frequency response of the simulated system
- The five types of motion profile, the operational motion profiles of the test records and the operating range of all records

~~In each channel $c \in \{X, \Theta, Y\}$ of the generalised force, the multisine is~~[own] ~~where $\Delta f = 0.005\,f_b$, $\mathcal{L} = \{212, \ldots, 594\}$ spans $1.06\,f_b$ to $2.97\,f_b$, and the phases $\alpha_{c,l}$ are drawn uniformly per record. It enters at the actuators as $u^{\mathrm{ff}} = P^{-1}u^{\mathrm{ms}}$.~~[own] ~~The grid places 18 lines inside the half-power width $2\zeta_a f_a = 0.09\,f_b$ of the anti-resonance, the narrowest feature of the system, and makes the multisine periodic in 2 s. Of 30 phase draws, the one with the lowest crest factor is kept; for Θ the crest factor is taken on the simulated yaw response. The rms amplitudes A_c keep every record within the force and stroke limits of the machine without scaling. K_1 is tuned for f_b on the rigid baseline at $Y = 0$. On the simulated system, the crossover of the loop lies between $0.75\,f_b$ and $1.06\,f_b$ over the axes and the training positions, so the band starts at the highest. The upper edge follows from the best linear approximation of the simulated system with friction under K_1, measured at the five training positions. Above the crossover, the band holds 90 % of the squared difference between this response and that of the baseline, counted where the difference exceeds its uncertainty. A rerun with the friction law of (37a) gives the same edges.~~[own]

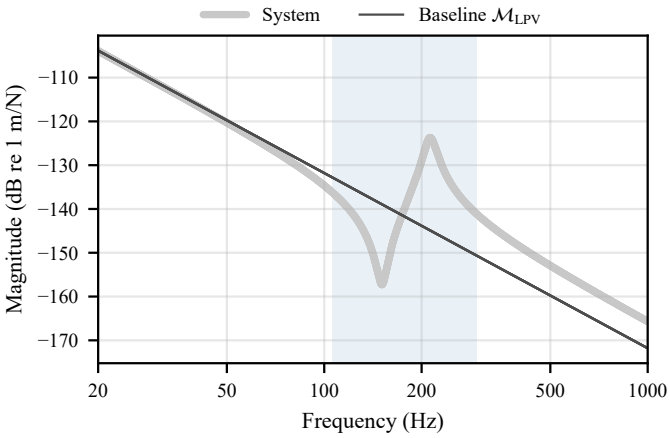


Fig. 11: Open-loop Bode magnitude from F_Y to Y of the system and the baseline at the five training positions. Shaded: multisine band.

~~The five types of motion profile are standstill at a fixed Y, slow sweeps of Y, X-Y Lissajous paths, point-to-point moves with S-curve profiles, and operational motion profiles, which are sine-shaped moves representative of machine operation.~~[moved to VI-B, own] ~~The test records I3 to I6 add new operational motion profiles: sine-shaped routes with X and Y together, alternating X-only and Y-only moves, an S-curve route and long strokes.~~[own] Table VIII lists the records~~, and Fig. ?? shows their operating range~~[own].

C.3 Processing and checks

- The reduction to the model rate: block mean of the force, zero-phase filter on the servo error, trim at the record ends
- The checks of rate, filter, window, controller rate and friction regime
- The cause of the controller-rate change (?)

~~With the argument counting samples at f_{sim}, the records are reduced to T_s by~~[own] ~~where $\mathcal{B}$ is an eighth-order Butterworth low-pass at 1.6 kHz, applied forward and backward.~~[own] **[TODO:** Changed from: where $\mathcal{B}$ is a zero-phase eighth-order Butterworth low-pass at 1.6 kHz.] **[Note:** Noise-free twins: a noise-free TRAINING run point-samples y, $y_k = y(5k)$ (config.py:33, data.py:175-176), but the evaluation loads the twins for the noise floor with the lowpass configuration (eval_blackbox.py:117, 156-158), so every reported floor uses the low-pass of Section VI-A as written. No final run used noise-free training data. Add a point-sampling sentence only if

TABLE VIII: Training, validation and test records.

Record	Motion	Multisine	New	Pair
Training	all five types	14 of 18		
Validation	all five types	4 of 6	positions, draws	
I1	point-to-point	yes	phases, setpoints	
I2	standstill	yes	position	
I3	operational	no†	motion type	
I4	route	no	motion type	
I5	route	no†	motion type	
I6	route	no†	motion type	
E1	standstill	yes	$\lvert Y\rvert = 1.3\,Y_{\mathrm{tr}}$	I2
E2	operational	no†	stroke of Y	I3
E3	route	no†	acceleration, 100 %	I5
E4	route	no†	velocity, 98 %	I6
E5	operational	no†	several, 131 % acc.	I3

I: interpolation, inside the training range. E: extrapolation, outside it in the setting under New, with the motion of the Pair record. New: what differs from the training records, for E from the Pair record. E5 differs in more than one setting and is a stress record. †Also ~~evaluated with the training multisine added~~ simulated with a multisine[own] (I3M, E2M, and so on), at equal amplitude within a pair and 0.30 of the training amplitude for I6 and E4.

TABLE IX: Checks of the experiment design.

Check	Value
Simulated loop, linearised at the training positions	
Phase-margin loss, 4 / 2 kHz	3.8° / 8.6°
Training records	
SNR 0.24 f_b to 1.05 f_b, without multisine	≥59 dB
K_1 at T_s, force error, median	0.5 %
largest; without multisine	1.0 %; 0.03 %
Force change within one hold of T_s, median	10.4 %

a noise-free training run is reported.] ~~The block mean is the exact input of a zero-order-hold model at T_s. The filter acts on the servo error only, so the reference passes unchanged. It changes the noise-free output by at most $2.3 \cdot 10^{-9}$ m RMS. Its impulse response falls below 1 % of its peak within 2.6 ms, so the 5 ms removed at both record ends hold its edge transient.~~[own]

~~$v_{\mathrm{fr}} = 1$ mm/s was chosen by simulating the loop with values between 0.1 and 10 mm/s. Higher values turn the friction into viscous damping under the multisine, and lower values keep the standstill servo error above the noise level. For $\lvert v_i\rvert \geq 3v_{\mathrm{fr}}$, the friction of (37a) is within 0.5 % of the Coulomb law.~~[own]

Table IX gives the checks of the design~~, including the share of samples in this range~~[own]. ~~The training loop re-discretises K_1 at T_s. At the same instants, its feedback force differs from that of the system by a median of 0.5 % RMS on the training records. Within one hold of T_s, the force of the system changes by a median of 10.4 % RMS, which is the cost of closing the loop at T_s in simulation.~~[own] ~~The residual form (15) equals the model loop of (14) up to one term. Let $\bar{u}_{\mathrm{act},k}$ be the force that K_{fb} computes from the recorded r_k and y_k, with state $\bar{x}_k^{\mathrm{fb}}$, plus u_k^{ff}. [derivation moved to Section IV-B] The first three terms are the model loop, which contains no noise after τ. The last term is the force error of K_1 at T_s above. After τ, the noise reaches the model only through this term.~~[moved to IV-B, own] ~~This force error is the last term of (16).~~[own]

TABLE X: Black-box search: the 11 configurations, each trained for at most 80 000 updates, and their best validation NRMS_e. Failed: the training loss became non-finite.

#	n_x	Width	LR	Param.	NRMS_e (%)	Best update	Status
0	8	8	$3 \cdot 10^{-3}$	7494	63.4	8000	completed
1	8	16	$3 \cdot 10^{-3}$	13422	59.9	28000	completed
2	8	32	$3 \cdot 10^{-3}$	27582	63.4	20000	completed
3	16	8	$3 \cdot 10^{-3}$	10270	52.5	50000	completed
4	16	16	$3 \cdot 10^{-3}$	16518	56.2	11000	failed
5	16	32	$3 \cdot 10^{-3}$	31318	54.8	75000	completed
6	32	8	$3 \cdot 10^{-3}$	16206	58.2	9000	completed
7	32	16	$3 \cdot 10^{-3}$	23094	57.5	3000	completed
8	32	32	$3 \cdot 10^{-3}$	39174	59.1	4000	completed
9	**16**	**16**	$1 \cdot 10^{-3}$	**16518**	48.2	**78000**	**completed**
10	16	16	$1 \cdot 10^{-2}$	16518	66.4	1000	failed

Bold: selected on validation (lowest NRMS_e; within 2 % the fewer parameters).

C.4 Black-box model

- The SUBNET model of the closed loop, from reference and injected force to servo error, with its encoder

The black box of Section VI-D is

$$x_{k+1}^{\mathrm{bb}} = f_{\mathrm{bb}}\big(\theta_{\mathrm{bb}}, x_k^{\mathrm{bb}}, r_k, u_k^{\mathrm{ms}}\big), \quad \hat{y}_k = r_k + h_{\mathrm{bb}}\big(\theta_{\mathrm{bb}}, x_k^{\mathrm{bb}}\big), \tag{46}$$

where $x_k^{\mathrm{bb}} \in \mathbb{R}^{n_x}$ and an encoder estimates x^{bb} at the window start from past inputs and servo errors, as in [15]. Its settings are the black-box rows of Table XI. Table X gives the configurations that were searched.[own]

C.5 Hyperparameters

~~Table XI lists the settings of every grey-box run and of the black-box search. The basis of a value is derived (computed in Section VI or Appendix C), inherited (from earlier work), tuned (on validation data), heuristic (fixed without a measurement) or compute-limited. Every model is trained once with one seed, so differences between models smaller than the variation between seeds are not resolved.~~[own]

TABLE XI: Hyperparameters of the identification.

Setting	Value
Model and data	
Model rate $1/T_s$	4 kHz
RK4 steps per T_s	1
Window n_f	400 (0.1 s)
Window stride	10 samples
Encoder window $n+1$	30 samples
Encoder start	least squares (SVD)
SVD cutoff (N windows, m regressors)	$\max(N, m)\,\epsilon$
Added states $n_{\bar{x}}$	0, 2, 8
Network, encoder correction	2×16, tanh
Precision	float64
Margin γ	0.99
Detuning of the combinations	10 %
Optimiser	
Adam rate, ϵ	10^{-5}, 10^{-16}
Batch	512 windows
Epochs	200
Validation interval	1300 updates
L-BFGS iterations, history	100, 50
L-BFGS inner steps, windows	20, 16384
L-BFGS tolerance (grad., change)	$2.6 \cdot 10^{-5}$, $2.6 \cdot 10^{-7}$
Seeds per model	1
PS (Section IV-E)	
S1 and S2 Adam rate	10^{-3}
S1 batch	64 windows
Decoder width	16
Residual horizon H	$n+1$
Left out records	last per class with at least 3 records
Check of the decoder	every 25 updates, 128 left-out windows
S1 length	100 to 350 updates
S1 stops improving	less than 2 % over 4 checks
S2 runs if the decoder explains	at least 10 %, also checked at update 150
S2 pairs $N_{\mathcal{P}}$ (fit, left out)	8192, 2048
S2 batch	512
S2 length	500 to 8000 updates
S2 stops improving	less than 1 % over 5 checks of 100 updates
Black box (Section VI-D)	
States n_x	8, 16, 32
Width of f_{bb}, h_{bb} (encoder $2\times$)	8, 16, 32
Adam rate	10^{-3}, $3 \cdot 10^{-3}$, 10^{-2}
Batch, stride	256, 25
Updates	80000
Selection	lowest validation NRMS_e